

Unit 5: MongoDB

Introducing NoSQL Databases

What are NoSQL databases?

NoSQL database is a type of database that stores and manages data differently from traditional relational (SQL) databases. The term NoSQL means “not only SQL” (a.k.a. non-SQL), showing that these databases offer other ways to work with data beyond just tables and rows.

Popular NoSQL databases include **MongoDB**, **Cassandra**, **Redis**, **Amazon DynamoDB**, **Neo4j**, **Elasticsearch**, and **Firebase Realtime Database**.

When should you use NoSQL databases?

NoSQL databases are ideal when:

- Data is dynamic or evolving, e.g., in applications like real-time analytics, content management, IoT, or social networks.
- Data that doesn't have a clear structure, such as images, videos, audio files, emails, or social media posts.
- Applications which require high throughput and low latency.
- Applications that may need to scale horizontally across many servers.

NoSQL databases are flexible. Based on the application's needs, they allow storing data in different formats like documents, key-value pairs, wide-columns, or graphs, without needing a fixed table structure. Known for their high performance, NoSQL databases are optimized for fast read and write operations, making them ideal for real-time applications. Additionally, these databases can scale easily without relying on a single, powerful server.

SQL vs. NoSQL: Key differences

Consider storing information about a person, such as their name and age.

In Relational (SQL) database data is stored in a table format, like a spreadsheet. Each row represents a person. Each column represents a field: ID, Name, and Age.

Example rows would be as follows:

- Row 1: ID = 1, Name = John Smith, Age = 23
- Row 2: ID = 2, Name = Bob Will, Age = 25

NoSQL representation

Where as in NoSQL databases, data is stored as JSON-like documents.

- **NoSQL document 1:** It's a simple document similar to the SQL row. The id, name, and age are stored as a single string.
- **NoSQL document 2:** This document shows nested data. The name field is broken into first and last, which is something SQL databases cannot do without creating another table.

```
{
  "id" : "1",
  "name" : "John Smith",
  "age" : "23"
}
```

NoSQL document 1

```
{
  "id" : "2",
  "name":
  {
    "first" : "Bob",
    "last": "Will"
  },
  "age" : "25"
}
```

NoSQL document 2

Visual understanding of relational database and NoSQL database

It means that in NoSQL databases, each document can have different fields, and new fields can be added to a document without affecting other documents. The table below summarizes the difference between SQL and NoSQL databases.

Feature	SQL databases (relational)	NoSQL databases
Data structure	Tables, rows, and columns	Documents, key-value, graph, etc
Schema	Fixed, predefined	Flexible, dynamic
Scaling	Vertical	Horizontal
Query language	SQL	Varies (often JSON-like or API)

Advantages and disadvantages of NoSQL

Based on the aforementioned concepts, the advantages of NoSQL databases are as follows:

- **Flexible data models:** These models can easily handle unstructured or semi-structured data.
- **Scalability:** NoSQL databases are designed to scale out across many servers.
- **Performance:** They are optimized for high-speed operations and large datasets.
- **Variety:** NoSQL databases come in multiple types (document, key-value, graph, column-family), allowing developers to choose the best fit for their use case.

NoSQL databases come with their own set of limitations, such as:

- **Lack of consistency:** Some NoSQL databases may temporarily show outdated data because they focus on speed and scalability.
- **No standard language:** Unlike SQL for relational databases, there isn't any standard way to query all NoSQL databases.
- **Complex transactions:** With NoSQL, handling multiple pieces of data can be challenging.

- **Less mature:** Some NoSQL databases are relatively new and may not have all the features or support of older, well-known databases.

However, NoSQL databases offer flexible and scalable solutions for modern data needs, especially when traditional relational databases fall short.

Introducing MongoDB and its Architecture

What is MongoDB?

MongoDB is a popular, open-source database designed for high performance, high availability, and easy scalability. Unlike traditional databases that store data in tables and rows, MongoDB stores data in flexible, JSON-like *documents*.

MongoDB is also one of the most widely used NoSQL databases. It is especially popular for web and mobile applications, real-time analytics, content management systems, and Internet of Things (IoT) applications.

Why should you choose MongoDB?

MongoDB is designed to handle a wide variety of data types and workloads, making it a great choice for many modern applications. It allows applications to grow without major architectural changes, as it adapts easily to changing data requirements. It supports a large community, extensive documentation, and many integrations.

Key features of MongoDB

Having learned about the core advantages of NoSQL databases already, let's take a look at the key features of MongoDB.

- **Dynamic data structures:** Unlike rigid table schemas, MongoDB collections can hold documents with varying fields and data types, allowing the data model to evolve as the application grows.
- **Effortless scalability:** Built with distributed systems in mind, MongoDB can automatically spread data across multiple servers, supporting applications that need to handle increasing loads or global reach.
- **Expressive queries and indexing:** MongoDB offers a powerful query language and supports a variety of indexes, enabling developers to efficiently search, filter, and analyze the data, even as it grows in size and complexity.
- **Seamless integration with modern development:** With its JSON-like document format and drivers for all major programming languages, MongoDB fits naturally into today's development workflows and technology stacks.

MongoDB's data structure

MongoDB organizes data differently from traditional relational databases. Instead of tables and rows, MongoDB uses documents and collections.

What is a document?

A **document** is the basic unit of data in MongoDB. Similar to JSON objects, documents store data as key-value pairs. They can store different types of data, including strings, numbers, arrays, and even nested documents. Basic example of a MongoDB document would be like :

```
{
  "_id": "63f2a45d5c8f9a1e4a6b987e",
  "name": "Alice",
  "age": 25,
  "email": "alice@example.com"
}
```

In the above example, the curly braces ({}) indicate that it is a document. Each piece of information is stored as a key-value pair, e.g., "name": "Alice". Also, every document has a unique identifier, `_id` (shown at **line 2**), that uniquely identifies each document and is never repeated.

Note: In MongoDB, the primary key must be named `_id`, which is a 12-byte hexadecimal value by default. We can still have other fields named `id` or anything else in our document.

What is a collection?

A **collection** is a group of MongoDB documents. It's similar to a table in SQL databases, but collections do not enforce a fixed schema. All documents in a collection are stored together, but they can have different structures. For example, suppose we have a collection called `person`:

```
1- {
2-   {
3-     "_id": "63f2a45d5c8f9a1e4a6b987e",
4-     "name": "Alice",
5-     "age": 25,
6-     "email": "alice@example.com"
7-   },
8-   {
9-     "_id": "63f2a45d5c8f9a1e4a6b987f",
10-    "name": "Bob",
11-    "age": 30,
12-    "email": "bob@example.com",
13-    "address": {
14-      "street": "123 Main St",
15-      "city": "Springfield"
16-    }
17-  },
18-   {
19-     "_id": "63f2a45d5c8f9a1e4a6b9880",
20-     "name": "Charlie",
21-     "skills": ["JavaScript", "Python"],
22-     "experience": 5
23-   }
24- }
```

Basic example of a MongoDB collection

It contains three documents (each enclosed in `{}`). Each document has a unique `_id` field. Documents in this collection have different structures:

- The first document (**lines 2–7**) has a basic structure (name, age, email).
- The second document (**lines 8–17**) has an additional address field (nested document).
- The third document (**lines 18–23**) has a skills array and an experience field, and no age or email.

What is a database?

A **database** in MongoDB is a container for collections. We can have multiple databases on a single MongoDB server, each storing collections for different applications or purposes.



Database as a large box containing several folders

Each database is independent and has its own collections and documents. A common practice is to use one database per application. For example, our database can be organized as:

```
1 myAppDatabase
2 └─ users (collection)
3 └─ products (collection)
4 └─ orders (collection)
```

A sample hierarchy

A sample hierarchy of a database is shown in the above figure.

Level	MongoDB term	Description	SQL equivalent
Top	Database	Container for collections	Database
Middle	Collection	Group of related documents	Table
Bottom	Document	Individual data entry (JSON-like)	Row/Record

In short, MongoDB's architecture—comprising databases, collections, and documents—offers a flexible and scalable way to organize data. This structure makes it easy to handle complex and evolving data models.

Using MongoDB Shell

What is the MongoDB Shell?

The MongoDB Shell (often called mongo or mongosh) is a command-line tool that allows us to interact directly with MongoDB server. It can be used to:

- Create and manage databases and collections.
- Insert, query, update, and delete documents.
- Run administrative commands.

Starting the MongoDB Shell

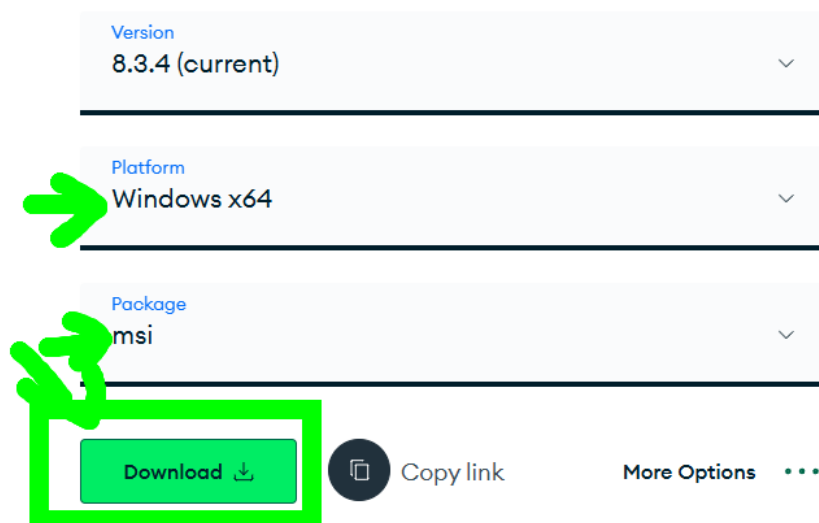
The basic prerequisite for mongoose to work is MongoDB installed on local machines.

Step 1: Download MongoDB

Visit the

<https://www.mongodb.com/try/download/community>

Next, choose your operating system: Windows, macOS, or Linux. Then download the installer. Try downloading the latest version of the MongoDB community server.



Step 2: Install and start MongoDB

For Windows

- **Run the installer:** Double-click the downloaded `.msi` file.
- **Follow the setup wizard:** Choose “Complete” setup. Optionally, install MongoDB Compass (a GUI for MongoDB).
- **Finish installation:** Click “Finish” when done.

MongoDB should start automatically as a Windows service. To start manually, open the command prompt and run:

net start MongoDB

For macOS

- **Use Homebrew (recommended):** Open terminal and run:

- `brew tap mongodb/brew`
- `brew install mongodb-community`

- **Manual installation:** Alternatively, use the downloaded `.tgz` file and follow the instructions on the MongoDB website.

To start MongoDB with Homebrew, type:

```
brew services start mongodb-community
```

For Linux (Ubuntu)

- **Import the public key:**

- `wget -qO - https://www.mongodb.org/static/pgp/server-6.0.asc | sudo apt-key add -`

- **Create a list file:**

- `echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu focal/mongodb-org/6.0 multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org-6.0.list`

- **Install MongoDB:**

- `sudo apt-get update`
- `sudo apt-get install -y mongodb-org`

Start the MongoDB server with:

```
sudo systemctl start mongod
```

Step 4: Verify installation

Open a terminal or command prompt and type:

mongod --version

Once installed, MongoDB Shell can be used by opening the terminal or command prompt and typing:

mongosh or **mongo** in older versions(till MongoDB version 4.4).

We should see a prompt like this:

```
1 test>
```

This means we are connected to the default database (test).

Note: MongoDB is case-sensitive. Collection names like Users and users are treated as two separate collections. Always use consistent casing to avoid confusion or unexpected results.

Basic MongoDB Shell commands

Below is a list of a few basic commands to get started.

- **Show databases:** List all databases on our server.
Syntax: `show dbs`
- **Switch/Create database:** Switch to a database (creates it if it doesn't exist).
Syntax: `use myDatabase`
- **Create a collection:** Create a new collection in the current database.
Syntax: `db.createCollection("users")`
- **Show collections:** List all collections in the current database. If none, it returns nothing.
Syntax: `show collections`

Understanding JSON and BSON

What is JSON?

JSON (JavaScript Object Notation) is a lightweight, human-readable data format for data exchange between applications. It is widely used in web development and APIs. Below is an example of JSON-formatted data, which we have already covered.

```
1 {  
2   "name": "Alice",  
3   "age": 25,  
4   "email": "alice@example.com"  
5 }
```

Sample of a JSON-formatted data

JSON is a text-based format that uses key-value pairs. Here, name, age, and email are keys. Their values are "Alice," "25," and "alice@example.com," respectively.

Note: JSON supports strings, numbers, arrays, booleans, and nested objects.

What is BSON?

BSON (Binary Javascript Object Notation) is a binary-encoded serialization format used internally by MongoDB to store documents, enabling efficient data transmission. It supports rich data types that aren't available in standard JSON, such as `Date`, `ObjectId`, and `Binary`.

It is faster to parse and generate, and it uses less space to store data. For example, BSON can store date values in a smaller, fixed size (8 bytes), while JSON would store the same date as a longer text string. This makes BSON more efficient for storing and processing date values.

While BSON is binary and not human-readable, it represents the same data as JSON. Here's an example.

```
\x16\x00\x00\x00      // total document size (22 bytes)
\x02                   // type: string
name\x00               // field name (key)
\x06\x00\x00\x00Alice\x00 // string value "Alice" (including length and null terminator)
\x10                   // type: int32 (integer)
age\x00                // field name (key)
\x19\x00\x00\x00      // integer value 25
\x02                   // type: string
email\x00              // field name (key)
\x13\x00\x00\x00alice@example.com\x00 // string value "alice@example.com" (including length and
null terminator)
\x00                   // end of document marker
```

BSON example showing how MongoDB stores data in a compact and binary format

Explanation:

- **Document size:** The first four bytes represent the total size of the document (22 bytes in this case).
- **Field types:**
 - \x02 (at **line 2**) indicates a string type for “name” and “email”.
 - \x10 (at **line 5**) indicates an integer type for “age”.
- **String values:** Strings store the length followed by the string content and a null character (\x00).
- **Integer value:** It is stored as a 4-byte (32-bit) integer.
- **End marker:** The document ends with a null byte (\x00).

JSON vs. BSON: Key differences

Feature	JSON	BSON
Format	Text-based	Binary
Readability	Human-readable	Not human-readable
Data types	Basic (string, number, etc)	Extended (ObjectId, Date, etc)
Usage in MongoDB	Input/output (API, shell)	Internal storage/transport

How MongoDB uses JSON and BSON

When interacting with MongoDB (e.g., via the shell or drivers), we use a JSON-like syntax. Internally, MongoDB converts JSON to BSON for storage and processing. When data is retrieved, it converts BSON back to JSON for display or transmission.

Few Perquisites for MongoDB Setup

Diagnosed PATH configuration and missing MongoDB tools installation

```
C:\Users\vishal reddy>mongod
'mongod' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\vishal reddy>mongosh
Current Mongosh Log ID: 6a4b824f365b420d180d818f
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.3.3
Using MongoDB:      8.3.4
Using Mongosh:       2.3.3
mongosh 2.9.2 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-07-08T19:36:21.493+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> exit

C:\Users\vishal reddy>mongoimport
'mongoimport' is not recognized as an internal or external command,
operable program or batch file.
```

Here's what's going on with each:

1. **mongod** not recognized

This is likely already running as a Windows service (that's why **mongosh** could connect without you starting it manually). But the **mongod.exe** binary itself isn't on your PATH. Find it — usually at: **C:\Program Files\MongoDB\Server\<version>\bin**

Add that folder to your PATH (see steps below), or just don't worry about it since it's running as a service anyway.

2. **mongo --version** not recognized

The old **mongo** shell was **removed** in recent MongoDB versions — it's fully replaced by **mongosh**, which you already have and is working. Use **mongosh --version** instead.

3. **mongoimport** not recognized

This one is different — **mongoimport**, **mongoexport**, **mongodump**, **mongorestore** are **not part of the main MongoDB Server install anymore**. They're bundled separately as "**MongoDB Database Tools**."

You need to download and install that separately:

👉 <https://www.mongodb.com/try/download/database-tools>

MongoDB Command Line Database Tools Download

The MongoDB Database Tools are a collection of command-line utilities for working with a MongoDB deployment. These tools release independently from the MongoDB Server schedule enabling you to receive more frequent updates and leverage new features as soon as they are available. See the [MongoDB Database Tools](#) documentation for more information.

Note: MongoDB Database Tools are distributed as open source (Apache 2.0), standalone products developed separately from the MongoDB Server.

Version

100.17.0

Platform

Windows x86_64

Package

msi

Download

Copy link

More Options

Fix for the PATH issue (once tools are installed)

Locate the bin folders, typically:

`C:\Program Files\MongoDB\Server\<version>\bin`

`C:\Program Files\MongoDB\Tools\<version>\bin` (for Database Tools)

Copy both paths.

Search "Environment Variables" in Windows Start menu → Edit the system environment variables → **Environment Variables**.

Under **System variables**, select **Path** → **Edit** → **New** → paste each path.

Click OK on all dialogs.

Close and reopen your Command Prompt (important — it won't pick up the change otherwise).

Test with `mongoimport --version`.

```
Command Prompt - mongod x + v
Microsoft Windows [Version 10.0.26200.8737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\vishal reddy>mongosh --version
2.3.3

C:\Users\vishal reddy>mongoimport
2026-07-06T18:37:41.915+0530 no collection specified
2026-07-06T18:37:41.915+0530 using filename `` as collection
2026-07-06T18:37:41.915+0530 error validating settings: invalid collection r

C:\Users\vishal reddy>mongod
{"t":{"$date":"2026-07-06T18:37:47.452+05:30"},"s":"I", "c":"-", "id":{"s","attr":{"seed":3838260317}}
{"t":{"$date":"2026-07-06T18:37:47.547+05:30"},"s":"I", "c":"CONTROL", "id":{"ng TLS 1.0 and TLS 1.1, to force-enable TLS 1.1 specify --sslDisabledProtocols 'none'}}
```

Data Portability and Setup

Importing and Exporting Data (CSV, JSON)

Why do we import and export data?

Importing allows us to load data into MongoDB (or any other database) from files created in other systems like spreadsheets, other databases, or APIs that export JSON or CSV. This is useful when we want to:

- Start a new project using existing data.
- Combine data from different sources into MongoDB.
- Quickly populate a collection for testing or development.

On the other hand, exporting lets us save MongoDB data into files that can be:

- Shared with teammates or stakeholders.
- Used for backups.
- Analyzed using external tools like Excel, Python, or R.

Import data into MongoDB

MongoDB supports a command-line tool called `mongoimport` that helps bring data into our database.

Syntax: `mongoimport [OPTIONS]`

Importing a JSON file

Download the `products.json` file by clicking the following link below:

https://drive.google.com/file/d/1dIFBoZnQaimfy5gh_CEqYijV15LAu8re/view?usp=drive_link

Let's say we want to import the `products.json` file. We can import it into a MongoDB database named `ecommerce` using this command:

Query:

`mongoimport --db ecommerce --collection products --file products.json --jsonArray`

Here:

- `--db ecommerce`: Use or create the `ecommerce` database.
- `--collection products`: Add the data to the products collection.
- `--file products.json`: This is the path to our data file.
- `--jsonArray`: This file contains a JSON array (i.e., multiple items in []).

Output:

```
C:\Users\vishal reddy\Downloads>mongoimport --db ecommerce --collection products --file products.json --jsonArray
2026-07-06T18:49:36.850+0530    connected to: mongodb://localhost/
2026-07-06T18:49:36.889+0530    100 document(s) imported successfully. 0 document(s) failed to import.
```

Importing a CSV file

Download the `products.csv` file by clicking the following link below:

Dataset:

<https://drive.google.com/file/d/1IvNeYENLjdHITxH4Tn8fDVPQ9m4b8gva/view?usp=sharing>

To import this:

```
mongoimport --db ecommercecsv --collection products --type csv --headerline --file products.csv
```

Here:

- `--type csv`: The file is in CSV format.
- `--headerline`: Use the first row as field names.
- `--file products.csv`: This is the file name.

Make sure our CSV headers match the field names we want in MongoDB.

Output:

```
C:\Users\vishal reddy\Downloads>mongoimport --db ecommercecsv --collection products --type csv --headerline
--file products.csv
2026-07-06T18:52:09.117+0530    connected to: mongodb://localhost/
2026-07-06T18:52:09.135+0530    100 document(s) imported successfully. 0 document(s) failed to import.
```

Export data into MongoDB

The `mongoexport` tool lets us save data from MongoDB collections to files in JSON or CSV format.

Syntax:

```
mongoexport [OPTIONS]
```

Exporting to a JSON file

Let's say we want to export our `orders` collection.

```
mongoexport --db ecommerce --collection products --out product.json --jsonArray
```

Here:

- `--out orders.json`: Output file name.
- `--jsonArray`: Export as an array of JSON objects.

Output:

```
C:\Users\vishal reddy\Downloads>mongoexport --db ecommerce --collection products
--out product.json --jsonArray
2026-07-06T18:53:46.632+0530    connected to: mongodb://localhost/
2026-07-06T18:53:46.642+0530    exported 100 records
```

Exporting to a CSV file

To export selected fields (**name** and **email**) from the users collection, in the **.csv** format; we can run the following command:

```
mongoexport --db ecommercecsv --collection products --type=csv --fields id,name --out product1.csv
```

Here:

- **--fields id,name**: Export only these columns.
- **--type=csv**: Export as CSV.

```
C:\Users\vishal reddy\Downloads>mongoexport --db ecommercecsv --collection
products --type=csv --fields id,name --out product.csv
2026-07-06T18:57:19.695+0530    connected to: mongodb://localhost/
2026-07-06T18:57:19.701+0530    exported 100 records
```

To Export the Entire csv file

Option 1: List every field manually with --fields

```
mongoexport --db ecommercecsv --collection products --type=csv --fields
"id,name,category,price,inStock,color,brand,material,rating" --out product2.csv
```

Note: nested fields (like your attributes sub-document) need dot notation — attributes.color, attributes.brand, attributes.material — otherwise they get skipped or dumped oddly.

Output:

```
C:\Users\vishal reddy\Downloads>mongoexport --db ecommercecsv --collection
products --type=csv --fields "id,name,category,price,inStock,color,brand,ma
terial,rating" --out product2.csv
2026-07-06T19:07:33.905+0530    connected to: mongodb://localhost/
2026-07-06T19:07:33.911+0530    exported 100 records
```

Option 2: Use --fieldFile (better if you have many fields or don't want to type them all)

1. Create a plain text file, e.g. fields.txt, with **one field name per line**:

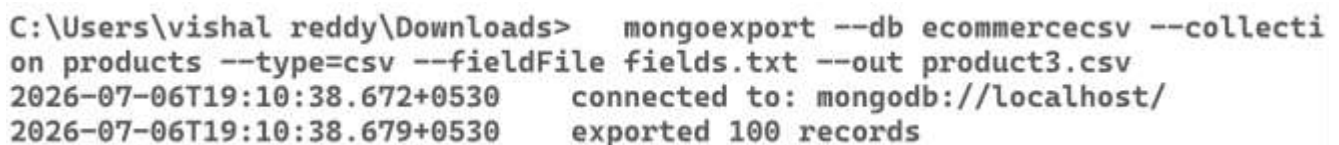
```
id
name
category
price
inStock
color
brand
material
rating
```



```
id
name
category
price
inStock
color
brand
material
rating
```

2. Run:

```
mongoexport --db ecommercecsv --collection products --type=csv --fieldFile fields.txt --out product3.csv
```



```
C:\Users\vishal reddy\Downloads> mongoexport --db ecommercecsv --collection products --type=csv --fieldFile fields.txt --out product3.csv
2026-07-06T19:10:38.672+0530    connected to: mongodb://localhost/
2026-07-06T19:10:38.679+0530    exported 100 records
```

If you genuinely don't know all the field names in advance

There's no built-in "auto-detect all fields" option in mongoexport. The common workaround:

- Export as **JSON first** (no --fields needed):

```
mongoexport --db ecommercecsv --collection products --out product.json
```

- Then convert that JSON to CSV using a script (like the one I wrote for you earlier) — that script can auto-flatten and pick up whatever fields exist, without you having to hardcode them.

Want me to also write that JSON→CSV conversion script as a standalone file you can reuse anytime, in case your collection's fields change later?

Backup and Restore Options

Why do we backup and restore data?

A **backup** is a saved copy of the database at a certain point in time. It protects from accidental deletions or overwrites, and avoids corrupted data due to software bugs, hardware failures (e.g., disk crash), and security incidents (e.g., ransomware). Without backups, we risk losing all our hard work permanently.

A **restore** brings the database back to a previous working state using the backup files. It's like clicking “Undo” on the entire database.

Backing up a MongoDB database

MongoDB provides the `mongodump` tool to create binary backups of our databases. This tool connects to a MongoDB instance, extracts all documents, and saves them as binary files in BSON format (not plain JSON). These files are saved on our local file system or a remote directory.

Let's see how to back up the `ecommerce` database:

```
mongodump --db ecommerce --out C:\backups\ecommerce-backup
```

Here:

- `--db ecommerce`: Choose the database we want to back up.
- `--out C:\backups\ecommerce-backup`: Choose the directory to save the backup files. MongoDB will create it if it doesn't exist.
- If you'd rather just dump it into your current folder (simpler, avoids path issues entirely):

```
mongodump --db ecommerce --out ecommerce-backup
```

Restoring a MongoDB database

MongoDB provides the `mongorestore` tool to restore data from a backup. This tool reads `.bson` and `.metadata.json` files created by `mongodump`. It inserts the data back into MongoDB—either replacing an existing database or creating a new one.

1. Connect to the MongoShell --- `mongosh`

```
C:\Users\vishal reddy\Downloads>mongosh
Current Mongosh Log ID: 6a4bb55a2b81a729de0d818f
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&se
rverSelectionTimeoutMS=2000&appName=mongosh+2.3.3
Using MongoDB:      8.3.4
Using Mongosh:       2.3.3
mongosh 2.9.2 is available for download: https://www.mongodb.com/try/downlo
ad/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

The server generated these startup warnings when booting
2026-07-05T19:36:21.493+05:30: Access control is not enabled for the dat
abase. Read and write access to data and configuration is unrestricted

test> |
```

2. We will Drop the database manually

```
test> use ecommerce
switched to db ecommerce
ecommerce> db.dropDatabase()
{ ok: 1, dropped: 'ecommerce' }
ecommerce> db.products.countDocuments()
0
ecommerce> exit
```

Now we will try to restore the `ecommerce` database from the backup.

```
mongorestore --drop --db ecommerce \backups\ecommerce-backup\ecommerce
```

```
C:\Users\vishal reddy\Downloads>mongorestore --drop --db ecommerce \backups
\ecommerce-backup\ecommerce
2026-07-06T19:34:01.366+0530 The --db and --collection flags are depreca
ted for this use-case; please use --nsInclude instead, i.e. with --nsInclud
e=${DATABASE}.${COLLECTION}
```

Here:

- `--db ecommerce`: This is the target database name in MongoDB. If it doesn't exist, MongoDB will create it.
- The path (`/backups/.../ecommerce`) should point to the folder containing `.bson` files.
- The `--drop` flag tells MongoDB to *drop (delete) each collection before restoring it*. This ensures that the restored data doesn't get mixed with any existing documents in the same collection.

Note: If the database already exists:

- **Without `--drop`:** MongoDB merges data into existing collections—new documents are added, but existing ones are not deleted.
- **With `--drop`:** MongoDB deletes the existing collections before restoring, ensuring a clean overwrite of data.

```
test> use ecommerce
switched to db ecommerce
ecommerce> db.products.countDocuments()
100
ecommerce> |
```

Basic CRUD Operations

Creating Databases and Collections

We don't need a special command to create a database in MongoDB. Simply, switch to a new database name using the `use` command as follows:

Syntax:-

```
use myNewDatabase
```

Remember that MongoDB has not created `myNewDatabase` yet. It will only be created when we add data to it, such as adding a collection or inserting a document into the collection. To create a collection, type:

```
db.createCollection("users")
```

This creates a collection named `users` in the current database. But there's no document inside it yet.

Now, `myNewDatabase` has a `users` collection in it and is ready for documents. Next, type the `show collections` command. It displays all collections added to the database.

Inserting documents

We can add one or more documents to a MongoDB collection at a time.

- To add a single document, use `insertOne()`:

```
db.users.insertOne({ name: "Alice", age: 25, email: "alice@example.com" })
```

Here, a single document is added to the `users` collection.

- To add several documents at once, use `insertMany()`:

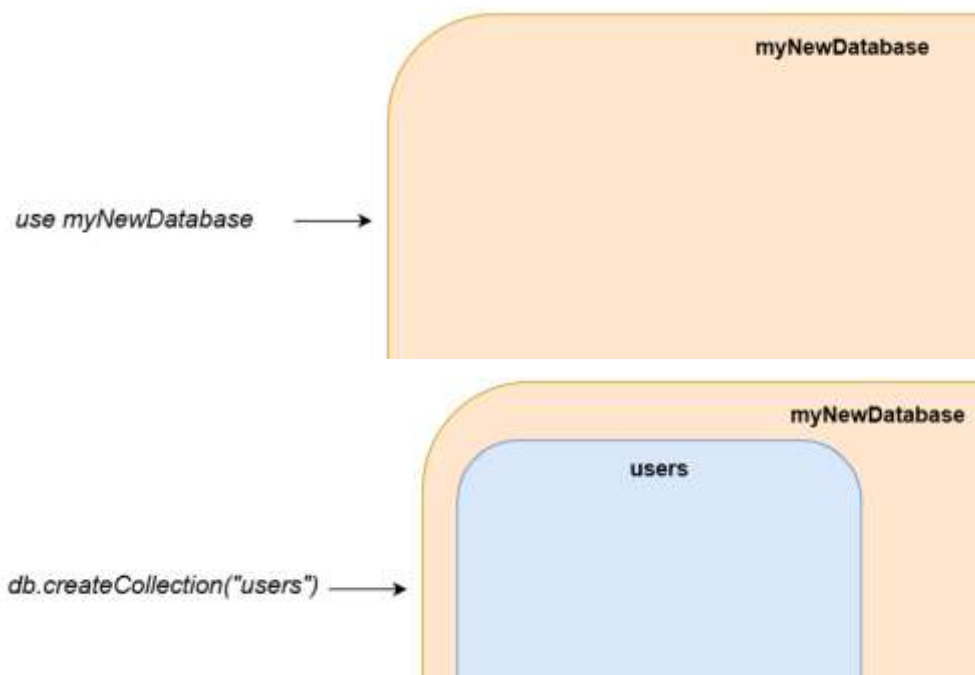
```
db.users.insertMany([
  { name: "Bob", age: 30 },
  { name: "Charlie", age: 28, email: "charlie@example.com" }
])
```

Here, we have inserted two documents at a time.

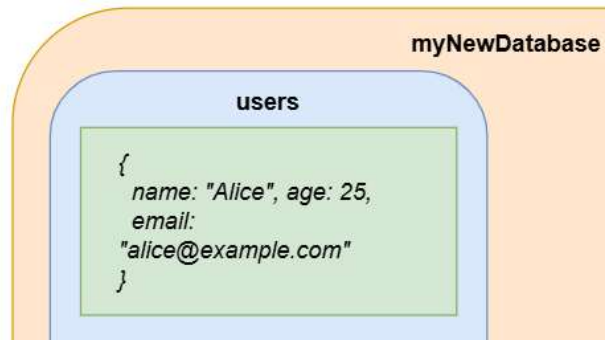
Try adding a single document as follows:

```
db.products.insertOne({ name: "Trackpad", category: "Computer devices" })
```

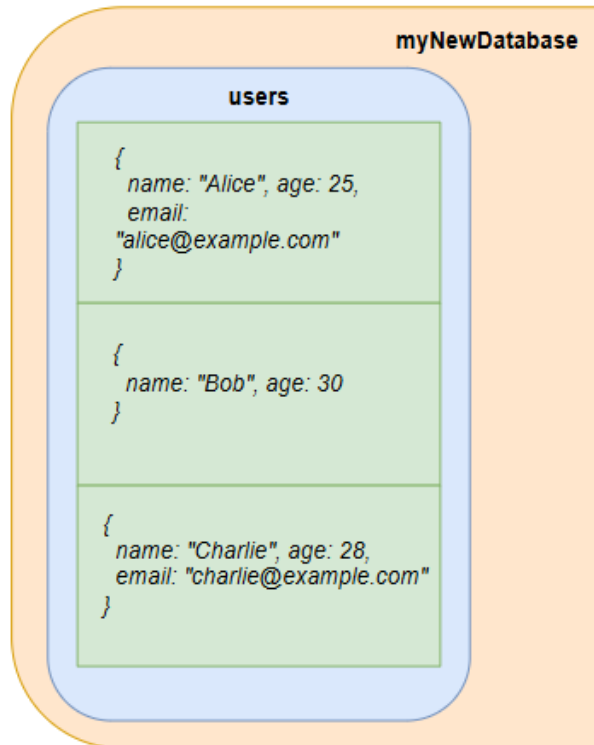
It directly creates the `products` collection and then adds the document to it. To verify, type the `show collections` command.



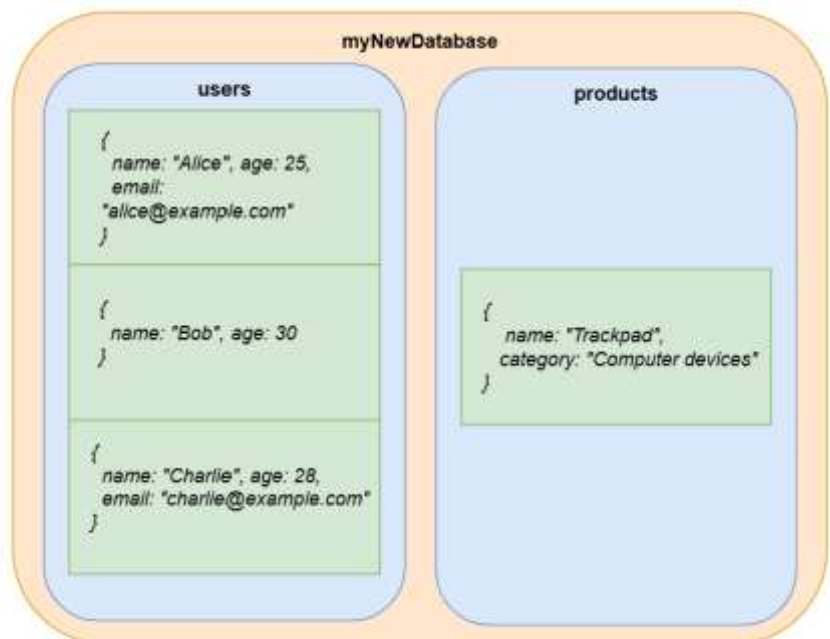
```
db.users.insertOne(  
  { name: "Alice", age: 25,  
    email: "alice@example.com" }  
)
```



```
db.users.insertMany([  
  { name: "Bob", age: 30 },  
  { name: "Charlie", age: 28,  
    email: "charlie@example.com" }  
)
```



```
db.products.insertOne(  
  { name: "Trackpad",  
    category: "Computer devices" }  
)
```



Comprehensive MongoDB Database: E-Commerce Platform

Database structure: E-commerce database

To understand the MongoDB basics, we'll get hands-on experience with a database for an e-commerce platform that sells products online. The database stores:

- Information about customers, e.g., their name, email, address, and order history.
- Products' details, e.g., their name, price, category, stock status, and ratings.
- Orders records, e.g., which user bought what and the mode of payment.
- Product reviews, e.g., who left the review against which product, and comments, if any.
- Product categories, e.g., organizes products into categories and subcategories.

The name of the database is **ecommerce**, and it has five collections to store the above information: **users**, **products**, **orders**, **reviews**, and **categories**.

Collections: Details and examples

The **users** collection stores customers' information, including login details, addresses, and their preferences (order history). Below is an link for the document:

Dataset link:

https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=drive_link

The **products** collection stores details about products, including varying attributes based on the product type.

Dataset link:

https://drive.google.com/file/d/1dIFBoZnQaimfy5gh_CEqYijV15LAu8re/view?usp=drive_link

The **orders** collection records order information for users, including items, status, and payment details.

Dataset link:

https://drive.google.com/file/d/1iKf1Ne_NHrK0OqAbfYAqV3HV1o_xVa5W/view?usp=drive_link

The **reviews** collection holds reviews for products left by users when using the application.

Dataset link:

https://drive.google.com/file/d/1LCKiYUQPq9ThPWhWm4P4s2pXktDJ7L9v/view?usp=drive_link

The **categories** collection organizes products into categories, with hierarchical relationships (graph-like structure).

Dataset link:

https://drive.google.com/file/d/1LKvPBMCsT2o5b0lBmqp-RKD0oOBaFpVR/view?usp=drive_link

Reading Data from Database

Reading data is a fundamental operation in any database management system. It allows us to access and retrieve information stored in the database, making it possible to view our data. By reading, we understand our database better, helping us make informed decisions and troubleshoot any data-related problems.

In MongoDB, the **.find()** method is the most common way to read data.

The **.find()** method

The **.find()** method is used to retrieve (read) documents from any MongoDB collection. Below are some common use cases for this method:

1. Finding all documents
2. Filtering the results
3. Applying a projection to the results

Syntax:

```
db.collection.find(query, projection)
```

Here, **collection** refers to the collection from which we want to get documents. **query** is an optional parameter, which is a condition for retrieving documents. **projection** is also an optional parameter that specifies which fields to include or exclude. Let's see these functionalities in detail.

Finding documents

The `.find()` method returns *all* documents from a collection if no query or projection is specified.

```
db.users.find()
```

This displays *all* the documents stored in the `users` collection. If you scroll through the complete output, you'll see that the output is truncated due to its large size. However, in real-world applications, simply displaying all documents is rarely useful. In practical scenarios, we won't need all the data in a collection; instead, we will be working with specific data.

Finding one document

The `.findOne()` method returns the first document from a collection if no query or projection is applied.

```
db.users.findOne()
```

This displays the first document stored in the `users` collection.

Filtering results

The `.find()` method allows us to filter documents on the basis of a certain criterion by adding a *query*.

Basic filtering

Suppose we want to get products with a rating of 4.9. We can use the `.find()` method as follows:

```
db.products.find({rating: 4.9})
```

Explanation: `db.products` specifies the `products` collection within the current MongoDB database. `{rating: 4.9}` is the query filter. It means that only documents where the `rating` field has a value `4.9` should be returned.

Try replacing `find()` with `findOne()` to observe that we will only receive the first document from the category that matches the criteria.

Here's another example. Consider retrieving users living in the city of San Francisco.

```
db.users.find({"address.city": "San Francisco"})
```

Explanation: `db.users` specifies the `users` collection within the current MongoDB database. And `{"address.city": "San Francisco"}` is the query filter. It means that only documents where the `city` field inside the `address` field has a value "San Francisco" should be returned.

Here `"address.city"` is an example of *dot notation* in MongoDB, which allows us to query nested fields within an object.

Using multiple conditions

What if we want to get information about products with a 4.9 rating that are out of stock so we can prioritize restocking them?

```
db.products.find({rating: 4.9, inStock: false})
```

Explanation: Here, two query filters are applied to the `products` collection. `rating: 4.9` specifies documents where the `rating` field has a value `4.9`. `inStock: false` specifies documents where the `inStock` field has a value `false`. As a result, the code above will return products that are out of stock and have a 4.9 rating.

Projection: Selecting specific fields

So far, every document in the output shows all its fields. But in real-world scenarios, we would want to see selective fields only. **Projection** lets us control which fields are included or excluded in the results.

Including fields

We can explicitly specify which fields to show. See the command below.

```
db.users.find(
  {},
  {name: 1, email: 1}
)
```

Explanation: We get only the name and email of *all* users in the output. In the code above, the empty braces (`{}`) mean no filtration is added to the `query`. For projection, `name: 1` and `email: 1` include the `name` and `email` fields, respectively.

What if we need to know which users rated which products, and what their ratings and comments were?

```
db.reviews.find(
  {},
  {userId: 1, productId: 1, rating: 1, comment: 1}
)
```

Explanation: `db.reviews` specifies the `reviews` collection within the current MongoDB database. We explicitly get the user ID, product ID, rating, and comment fields from every document in this collection.

Excluding fields

Now, imagine there are n fields in a collection, and we want to return $n - 1$ fields. Would it be easier to include $n - 1$ or exclude that one field? Definitely, it's the latter. Let's see an example.

```
db.users.find(  
  {},  
  {password: 0}  
)
```

This time, we get complete information on users, except for their `password`.

By default, MongoDB always includes the `_id` field in the query results. Specifying `_id: 0` in the projection explicitly hides the `_id` field from the results. Feel free to try adding this projection to the widget above.

Mix projection is not allowed

MongoDB needs to maintain a clear and consistent approach when projecting fields. Mixing inclusion and exclusion creates ambiguity about which fields to include or exclude.

For example:

```
db.products.find(  
  {},  
  {name: 1, price: 0}  
)
```

This query is invalid because we're trying to include (1) the `name` field, and at the same time, exclude (0) the `price` field. MongoDB cannot resolve this conflict because it is unclear whether we want to include or exclude fields.

The only exception: `_id` field

The only time we can mix inclusion and exclusion (also known as dynamic projection) is with the `_id` field. This is because the `_id` field is included by default, and MongoDB allows us to explicitly exclude it without breaking the inclusion/exclusion rule. Try the query below and note the output.

```
db.users.find(  
  {},  
  {name: 1, email: 1, _id: 0}  
)
```

Optimizing Query Results: Limit and Skip

Limiting results in MongoDB

The `.find()` method returns all the matching documents that satisfy the provided query criteria. While this course's database is relatively small, real-world databases can contain thousands or even millions of documents. Imagine receiving thousands of documents, where only the top 10 records were required. To resolve this, MongoDB provides two methods to control the number of documents: `.limit()` and `.skip()` methods.

The `.limit()` method

The `.limit()` method allows us to specify the maximum number of documents that a query should return. This is useful for cases where we only need a subset of our data.

Syntax:

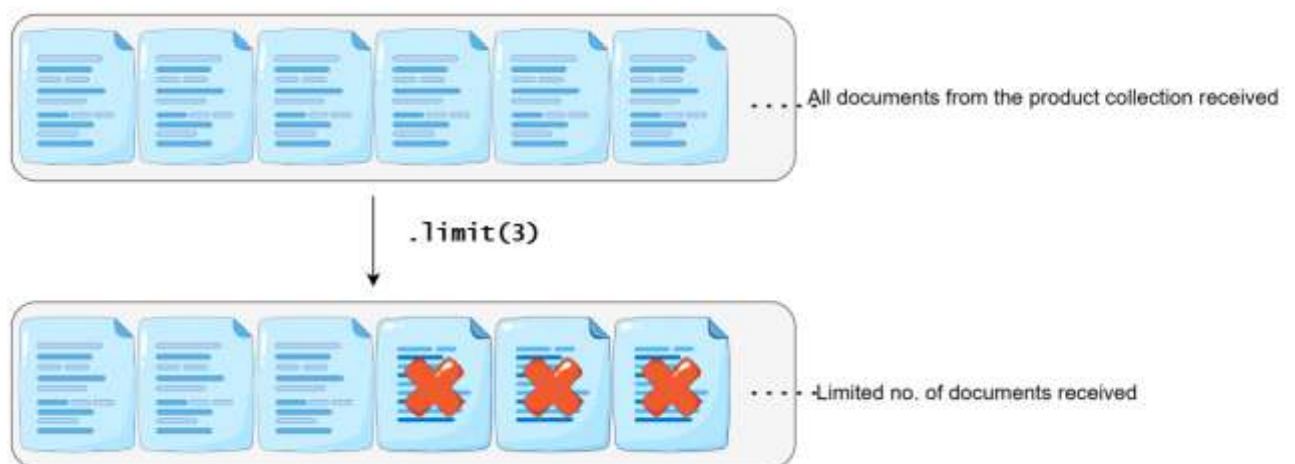
```
db.collection.find().limit(N)
```

Here, **N** is the number of documents we want to retrieve. If **N** is 0 or a negative value, MongoDB will not limit the results.

Below is a simple query example.

```
db.products.find().limit(3)
```

Explanation: Notice the output. The `db.products.find()` fetches all documents. However, by chaining the `.limit(3)` method, we restrict the results to only the first three documents.



Reading documents with the `.limit()` method chained

Look at another example below.

```
db.products.find({rating:4.9}).limit(2)
```

Explanation: The `db.products.find({rating:4.9})` fetches all products where the `rating` is 4.9. Once matching documents are found, the `limit(2)` ensures that only the first two matched documents are returned.

The `.skip()` method

The `.skip()` method allows us to ignore a specified number of documents from the beginning of the result set. This is useful for creating pages of results.

Syntax:

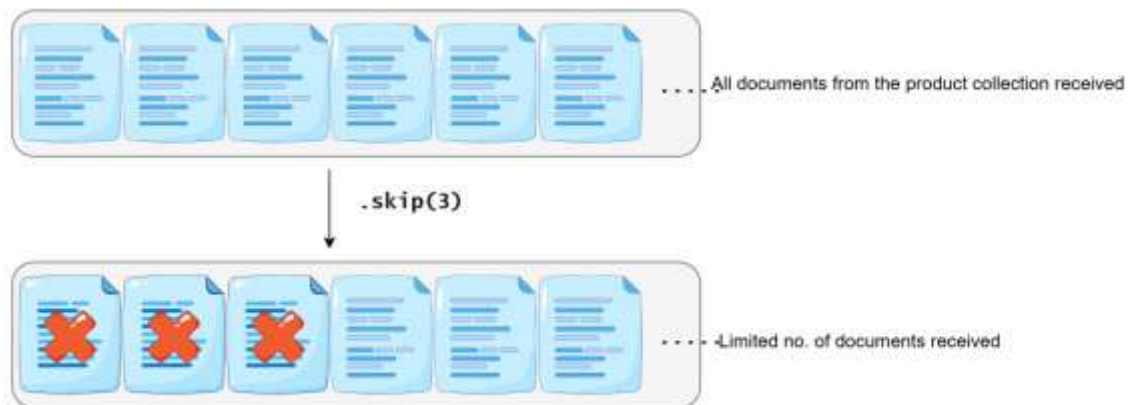
```
db.collection.find().skip(N)
```

Here, `N` is the number of documents we want to skip. If `N` is 0 or a negative value, MongoDB will not limit the results. Also, if `N` is greater than (or equal to) the total number of documents, no results are returned.

Below is a simple query example.

```
db.products.find().skip(3)
```

Explanation: Notice the output. The `db.products.find()` fetches all documents. However, by chaining the `.skip(3)` method, we omit the first three documents and return the rest.



Reading documents with the `.skip()` method chained

Look at another example below.

```
db.orders.find({status:"Shipped"}).skip(5).limit(5)
```

Explanation: The `db.orders.find({status:"Shipped"})` fetches all the orders that were shipped. Once matching documents are found, the `skip(5)` method ensures that the first five matched documents are skipped. Then, `limit(5)` returns the next five documents from the remaining matched documents.

With `.limit()` and `.skip()`, we're now equipped to handle large result sets efficiently and build powerful, paginated queries in MongoDB.

Sorting Query Results

Sorting in MongoDB

Sorting is an essential technique for organizing query results in a specific order, making them easier to analyze by focusing on prioritized information. It is a fundamental part of data retrieval because it helps us:

- Arrange data in a logical order (e.g., alphabetical, numerical).
- Highlight important records (e.g., highest-rated products).

Basic sorting with `.sort()`

MongoDB allows us to sort query results using the `.sort()` method.

Syntax:

// Ascending order

```
db.collection.find(query).sort({ field: 1 })
```

// Descending order

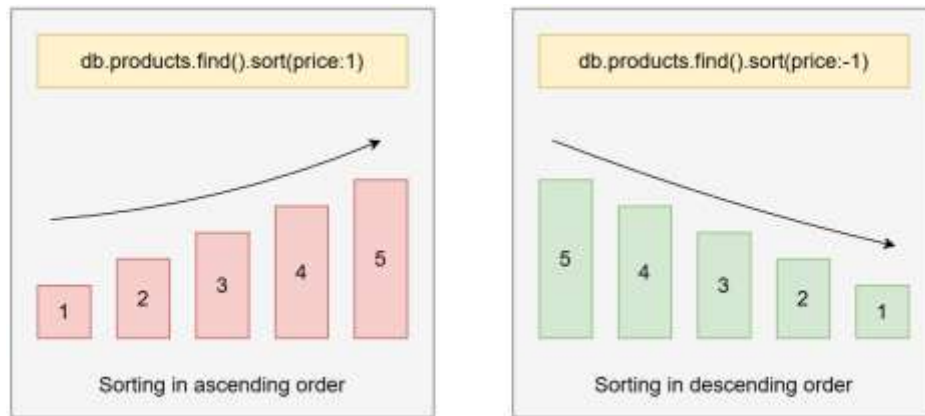
```
db.collection.find(query).sort({ field: -1 })
```

- Sorting in ascending order, `{ field: 1 }`, arranges values from smallest to largest (A-Z, 0-9).
- Sorting in descending order, `{ field: -1 }`, arranges values from largest to smallest (Z-A, 9-0).

For example, suppose we want to sort products by price, i.e., lowest to highest.

```
db.products.find(  
  {},  
  {name:1, price:1}  
) .sort({ price: 1 })
```

Explanation: Notice the output. The `db.products.find()` fetches all documents as `query` is empty. For better readability, we applied a projection to include `name` and `price` only. Next, by chaining the `.sort({price:1})` method, the results get sorted in ascending order. In other words, products are now displayed from the cheapest to the most expensive.



Visual explanation of how .sort() works

Look at another example below.

```
db.users.find(  
  {},  
  {name:1}  
)sort({ name: -1 })
```

Explanation: The query above sorted the users' documents in descending order on the basis of their names, which means the user with the last name in alphabetical order (Z) appears first, and the user with the first name in alphabetical order (A) appears last.

Sorting by multiple fields with .sort()

MongoDB allows us to sort by multiple fields. This is useful when we want to further refine the sorted results. Suppose we want to retrieve the most expensive products with lower ratings.

```
db.products.find(  
  {},  
  {rating:1, price:1}  
)sort({ rating: 1, price: -1 })
```

Explanation: Here, the first field sorts the results in ascending order based on rating (lowest to highest). For products with the same rating, **price: -1** further sorts them by price in descending order (from most expensive to least expensive).

Note that the order of fields matters in the .sort() method. Here, by sorting, we display the products with lower ratings, but prioritize those with higher prices among products of the same rating.

Querying with Regular Expressions

What are regular expressions?

Regular expressions, also known as regex, are special text strings that define a *search pattern*. They can be used to search, match, and manipulate text within documents. MongoDB supports JavaScript-style regular expressions, making them both familiar and highly versatile.

Syntax:

```
db.collection.find({ field: /pattern/ })
```

Here, **field** is the name of the field we want to search, and **/pattern/** is the regex pattern we want to match. The pattern is enclosed between **/** slashes.

Components of a regular expression

Regex are not just random characters—they are a precise language for defining search patterns in text. At their core, regular expressions are made up of three main components:

1. Literal characters
2. Metacharacters
3. Flags

Let's take a look at them in detail below:

- **Literal characters:** They are the exact characters we want to match in the text. For example, **/apple/** matches any string containing “apple”.
- **Metacharacters:** They are special symbols that give regex its power. They allow us to specify patterns instead of exact text. Only a few of them are listed below.
 - The **^** metacharacter matches the start of a string. For example, **/^apple/** matches any string that starts with “apple”.
 - The **\$** metacharacter matches the end of a string. For example, **/apple\$/** matches any string that ends with “apple”.
 - The **.** metacharacter matches any single character. For example, **/a.b/** matches “aab”, “acb”, and “alb”.
 - The ***** metacharacter matches zero or more occurrences of the previous character. For example, **/go*gle/** matches “ggle,” “gogle,” “google,” and “gooogle”.
 - The **+** metacharacter matches one or more occurrences of the previous character. For example, **/go+gle/** matches “gogle,” “google,” and “gooogle” but not “ggle”.

- The `|` metacharacter acts as an OR condition. For example, `/apple|banana/` matches any string containing “apple” or “banana”.
- **Flags:** These are optional modifiers that change the behavior of our regex search. For example, the `i` flag makes the search case-insensitive.

Examples

For example, we want to get users whose email addresses match a specific domain. Take a look at the query below.

```
db.users.find({ email: /gmail\.com$/ })
```

Here, the pattern is `/gmail\.com$/`. This matches any email that ends with “gmail.com” only. In regular expressions, a plain dot (`.`) means any character. If we want to match a literal dot (like the one in “gmail.com”), we have to escape it with a backslash as `\.`

Let's take a look at another example.

```
db.users.find({ name: /^[A-Z]/ })
```

This pattern matches any username that starts with an uppercase letter (A-Z).

Let's look at another example.

```
db.users.find({ phoneNo: /^\d{3}-\d{3}-\d{4}$/ })
```

This matches phone numbers in the format "123-456-7890".

- `^\d{3}`: Three digits at the start
- `-`: A dash
- `\d{3}`: Another three digits
- `\d{4}$`: Four digits at the end

We can combine regex with other query operators for more complex conditions. For example:

```
db.users.find({
  name: /^A/,
  email: /icloud\.com$/i
})
```

The query above finds users whose name starts with “A” and whose email ends with “icloud.com” (case-insensitive). Here, the `i` flag makes the search case-insensitive. It will match “icloud.com,” “Icloud.com,” “ICLOUD.COM,” and any other capitalization.

Advanced CRUD Operations

Basic Query Operators: Comparison and Logical Operators

In MongoDB, **query operators** are special symbols or keywords that allow us to specify conditions for matching documents in our database. They help create more flexible and powerful queries to filter and retrieve data.

There are three basic types of query operators:

1. **Comparison operators:** They filter documents based on value comparison.
2. **Logical operators:** They combine multiple conditions in a query.
3. **Element operators:** They check for the existence of fields or their data types.

They're used within the `.find()` method and applied to the field's value directly. These operators are always prefixed with a `$` symbol. Let's look at them in detail below.

Comparison operators

In real-world scenarios, the database is normally large, and we do not always want to see all the documents. Depending on the use case, we require selected documents only. **Comparison operators** let us filter documents based on the values of fields. Here are some of the most commonly used comparison operators:

Operator	Description
<code>\$eq</code>	Matches values equal to a specified value
<code>\$gt</code>	Matches values greater than a specified value
<code>\$gte</code>	Matches values greater than or equal to a specified value
<code>\$lt</code>	Matches values less than a specified value
<code>\$lte</code>	Matches values less than or equal to a specified value
<code>\$ne</code>	Matches values not equal to a specified value

Orders.json:

https://drive.google.com/file/d/1iKf1Ne_NHrK0OqAbfYAqV3HV1o_xVa5W/view?usp=sharing

Products.json:

https://drive.google.com/file/d/1dIFBoZnQaimfv5gh_CEqYijV15LAu8re/view?usp=sharing

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Reviews.json:

[https://drive.google.com/file/d/1LCKiYUQPq9ThPWhWm4P4s2pXktDJ7L9v/view?usp=drive link](https://drive.google.com/file/d/1LCKiYUQPq9ThPWhWm4P4s2pXktDJ7L9v/view?usp=drive_link)

Queries:

```
db.users.find({"address.city": {$eq: "San Francisco"}}).limit(2)
```

```
db.products.find({price: {$gt: 300}}).limit(2)
```

```
db.reviews.find({rating: {$lte: 3}}).limit(2)
```

```
db.orders.find({status: {$ne: "Delievered"}}).limit(2)
```

Explanation:

- **Line 1:** `{"address.city": {$eq: "San Francisco"}}` returns users who live in San Francisco. It searches for the documents where the `city` field inside the `address` field has a value "San Francisco." This is equivalent to `{"address.city": "San Francisco"}` which we covered in the previous lesson.
- **Line 3:** `{price: {$gt: 300}}` returns products with a price greater than 300.
- **Line 5:** `{rating: {$lte: 3}}` returns documents from `reviews` where users rated a product less than or equal to 3.
- **Line 7:** `{status: {$ne: "Delievered"}}` returns orders that have not been delivered yet.

Logical operators

Logical operators allow us to combine multiple queries in a single query. In other words, they allow us to search for documents that match one or more criteria. Here are the most commonly used logical operators:

Operators	Description
\$and	Matches documents that satisfy all conditions
\$or	Matches documents that satisfy any condition
\$not	Inverts a condition
\$nor	Matches documents that do not match any condition

Orders.json:

https://drive.google.com/file/d/1iKflNe_NHrK0OqAbfYAqV3HV1o_xVa5W/view?usp=sharing

Products.json:

https://drive.google.com/file/d/1dIFBoZnQaimfy5gh_CEqYijV15LAu8re/view?usp=sharing

Queries:

```
db.products.find({ $or: [
  { inStock: false },
  { rating: { $lt: 3 } }
] }).limit(2)
```

```
db.products.find({ $and: [
  { inStock: true },
  { price: { $gt: 50 } }
] }).limit(2)
```

```
db.products.find({ inStock: { $not: { $eq: false } } }).limit(2)
```

```
db.users.find({ $nor: [
  { "address.city": "Seattle" },
  { "address.city": "Jacksonville" }
] }).limit(2)
```

Explanation:

- **Lines 1–4:**
 - The `{ inStock: false }` query returns products that are out of stock.
 - The `{ rating: { $lt: 3 } }` query returns products that have a rating below 3.
 - The `{ $or [ { inStock: false }, { rating: { $lt: 3 } ] }` returns products that are either out of stock or have a rating below 3.
- **Lines 6–9:**
 - The `{ inStock: true }` query returns products that are in stock.
 - The `{ price: { $gt: 50 } }` query returns products priced above \$50.
 - The `{ $and [ { inStock: true }, { price: { $gt: 50 } ] }` returns products that are in stock and priced above \$50.
- **Line 13:** The query excludes products that are out of stock. Note that `{ $not: { $eq: false } }` actually returns `true`.
- **Lines 16–19:** The query retrieves users who are not from Seattle or Jacksonville. In other words, it will return all users except those from Seattle or Jacksonville.

Basic Query Operators: Element Operators

Element operators are special query operators in MongoDB that allow us to:

- Check if a field exists in a document.
- Verify the data type of a field.

Note: Do you remember that MongoDB is a schema-less database? It means documents in the same collection can have different/optional fields.

Here are the two most commonly used logical operators: `$exists` and `$type`.

The `$exists` operator

The `$exists` operator checks if a specified field is present in a document. The following query shows this.

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query:

```
db.users.find(
```

```
{ orderHistory: { $exists: false }  
})
```

Explanation: The `{ orderHistory: { $exists: false } }` query tells the `.find()` method to look for the documents where the `orderHistory` field is missing. In simple terms, it displays only those users who have never placed an order.

The `$type` operator

The `$type` operator checks if a specified field is of a certain data type. It is useful for ensuring data consistency, especially when fields can store values of multiple types.

For example:

Products.json:

https://drive.google.com/file/d/1dIFBoZnQaimfy5gh_CEqYijV15LAu8re/view?usp=sharing

```
db.products.find(  
  { rating: { "$type": "double" }  
})
```

Explanation: The `{products: { "$type": "double" } }` query tells the `.find()` method to look for the documents where rating is a floating-point number. Try replacing `double` with `int`, and notice the change in the output.

Updating and Deleting Documents

Updating documents in MongoDB

In MongoDB, data is not static. Applications will often need to update existing data. For example, a user may change their email address, a product's price may need to be adjusted, or an order's status may change from "Pending" to "Shipped".

MongoDB provides the following two powerful methods to update data in a collection:

- `.updateOne()`: Updates a single matching document
- `.updateMany()`: Updates all matching documents

Note: The scope of modifications made to the data in this lesson is confined to the executables only. The changes won't reflect elsewhere. We'll see variations of the `.find()` method to read data after those modifications to notice the change.

The `.updateOne()` method

The `.updateOne()` method allows us to update *only* the first document that matches a given filter. It is perfect for situations where we only want to change a single record.

Syntax:

```
db.products.updateOne( query, update)
```

Here, `query` refers to a criterion that matches the document we want to modify, and `update` refers to an update operation to apply. Let's take a look at an example below.

Products.json:

https://drive.google.com/file/d/1dIFBoZnQaimfy5gh_CEqYijV15LAu8re/view?usp=sharing

Query: Update the price of a single product using `updateOne()` and `$set`

```
db.products.updateOne(
  { id: "2001" },
  { $set: { price: 200 } }
),
// Display the change
db.products.find({ id: "2001" }, {id:1, price:1})
```

Explanation:

- `{id: "2001"}`: This query finds the first document in the `products` collection where `id` is "2001".
- `{ $set: { price: 200 } }`: This update uses the `$set` operator to update the price to 200.

Imagine another scenario, where a customer logs in by entering their email and password, and then wants to change their address.

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query: Update the address of a user using `updateOne()` and `$set`

```
db.users.updateOne(
  { email: "julie.bird362@hotmail.com" },
  { $set: { "address.street": "803 Snyder Mills Suite 473" } }
)
```

Explanation: We receive an error as follows: `Cannot create field 'street' in element {address:...}`. Recall that the `address` field is an array of objects. We'll study arrays in upcoming lessons. For now, just remember that this is not the correct method to update arrays with the `$set` operator.

The `.updateMany()` method

The `.updateMany()` method allows us to update all documents that match the provided criterion. It is ideal for bulk updates. Its syntax is identical to the `.updateOne()` method.

Imagine the management decides to stop selling products of the 'Home & Kitchen' category. The following query can perform this operation.

Products.json:

https://drive.google.com/file/d/1dIFBoZnQaimfy5gh_CEqYijV15LAu8re/view?usp=sharing

Query: Mark all “Home & Kitchen” products as out of stock using `updateMany()` and `$set`

```
db.products.updateMany(
  { category: "Home & Kitchen" },
  { $set: { inStock: false } }
),
```

// Display the change

```
db.products.find({ category: "Home & Kitchen" }, {id:1, name: 1, inStock:1})
```

Explanation: This query finds all the documents in the `products` collection where the `category` is “Home & Kitchen”. It uses the `$set` operator to update the `inStock` field to `false`.

Adding a new field

In MongoDB, it is possible to add a new field to existing documents. For example, in our case, we may decide to introduce a premium membership feature, distinguishing between premium and non-premium users. To achieve this, we can easily add a new field, `isPremium`, to the user documents.

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query: Add an isPremium field with value 'false' to all users using updateMany()

```
db.users.updateMany(
  {},
  { $set: { isPremium: "false" } }
),
// Display the change
db.users.find().limit(5)
```

Explanation:

- Here, the **query** is missing, which implies that an update will be made to all user documents.
- The **\$set** operator adds a new field, **isPremium**, to all the documents and sets the value to **false** by default.

Note: We cannot have an empty value in the **\$set** operator. If it goes missing (e.g., **{ \$set: { isPremium} }**), the query will return a syntax error.

What if we are not sure about the values when adding a new field? In that case, we simply pass **null** as: **{ \$set: { isPremium:null} }**.

Note: We can similarly use the **.updateOne()** method to add a field to a specific user only.

Removing a field

Take a look at the query below:

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query: Remove the phoneNo field from all user documents using \$unset

```
db.users.updateMany(
  {},
  { $unset: { phoneNo: "" } }
),
```

```
// Display the change
db.users.find().limit(5)
```

Explanation:

- Here, the `query` is missing, which implies that an update will be made to all user documents.
- Note, we have used a new operator: `$unset`. It is not for creating fields, but for removing them. The `{ phoneNo: "" }` part specifies to delete the `phoneNo` field from every document, regardless of its value.

Deleting documents in MongoDB

In MongoDB, we may need to remove documents that are:

- Outdated (old records).
- No longer needed (inactive users).
- Incorrect or duplicated (data cleanup).

The `.deleteOne()` method

The `.deleteOne()` method removes only the first document that matches the filter.

Syntax:

`db.users.deleteOne(query)`

Here, the `query` specifies the condition used to identify the first document to be deleted based on the collection's natural order. For example:

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query: Delete a single user document where the name is 'Olivia Moore' using `deleteOne()`

```
db.users.deleteOne({ name: "Olivia Moore" }),
```

```
// Display the change
```

```
db.users.find({name:"Olivia Moore"})
```

Explanation: This query deletes a user named “Olivia Moore.”

The `.deleteMany()` method

The `.deleteMany()` method removes all the documents that match the `query`. Its syntax is identical to the `.deleteOne()` method.

Imagine the management decides to remove the data of all cancelled orders from the `orders` collection. The following query can do this.

Products.json:

https://drive.google.com/file/d/1dIFBoZnQaimfy5gh_CEqYijV15LAu8re/view?usp=sharing

Query: Delete all orders with status 'Cancelled' using deleteMany()

```
db.orders.deleteMany({ status: "Cancelled" }),
```

```
// Display the change
```

```
db.orders.find({status:"Cancelled"})
```

This query removes all the documents in the `orders` collection where the `status` is “Cancelled”.

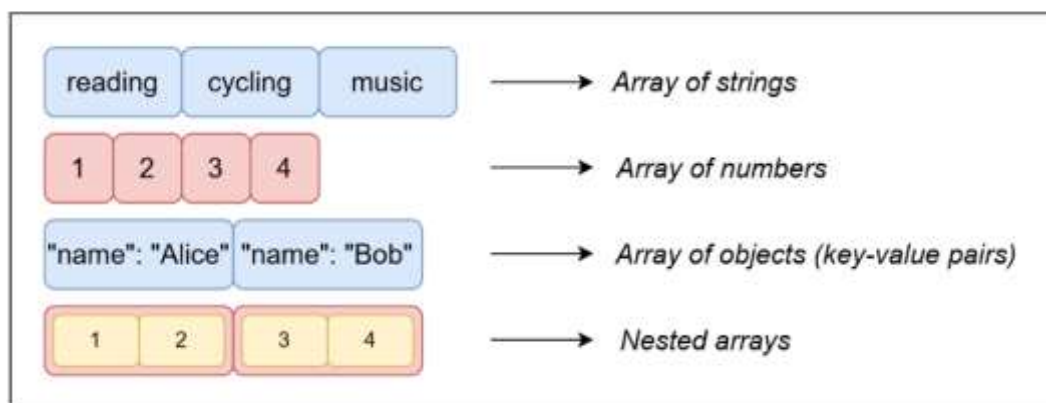
Reading Arrays from Documents

Understanding arrays in MongoDB

In MongoDB, an **array** is a list of values within a field of a document. These values can be of any data type:

- **Strings:** `["reading", "cycling", "music"]`
- **Numbers:** `[1, 2, 3, 4]`
- **Objects:** `[ { "name": "Alice" }, { "name": "Bob" } ]`
- **Nested arrays:** `[ [1, 2], [3, 4] ]`

Visually, we can see them as below.



Arrays in MongoDB

A MongoDB document can have multiple values for a single field. In our case, the `users` collection contains a field, `orderHistory`, which is an array of objects. See the snippet below.

```
{
  "id": "0001",
  "name": "Allison Hill",
  "email": "allison.hill546@icloud.com",
  "password":
"c9bc39914ae4497ea94dc59182b7dfe204445f817e4a880f259b1c29e2046a4f",
  "phoneNo": "985-559-7734",
  "address": [
    {
      "street": "26542 Susan Junction Apt. 161",
      "city": "Washington",
      "postalCode": "20001",
      "country": "United States"
    }
  ],
  "orderHistory": [
    {
      "id": "3000"
    },
    {
      "id": "3001"
    },
    {
      "id": "3002"
    },
    {
      "id": "3003"
    }
  ]
}
```

In the document above, the `orderHistory` field has an array, which contains four objects: `"id": "3000"`, `"id": "3001"`, `"id": "3002"`, and `"id": "3003"`.

Let's take a look at another use case. The `categories` collection contains a field, `subCategories`, which is also an array of objects.

```
{
  "_id": {
    "id": "5000"
  },
  "name": "Electronics",

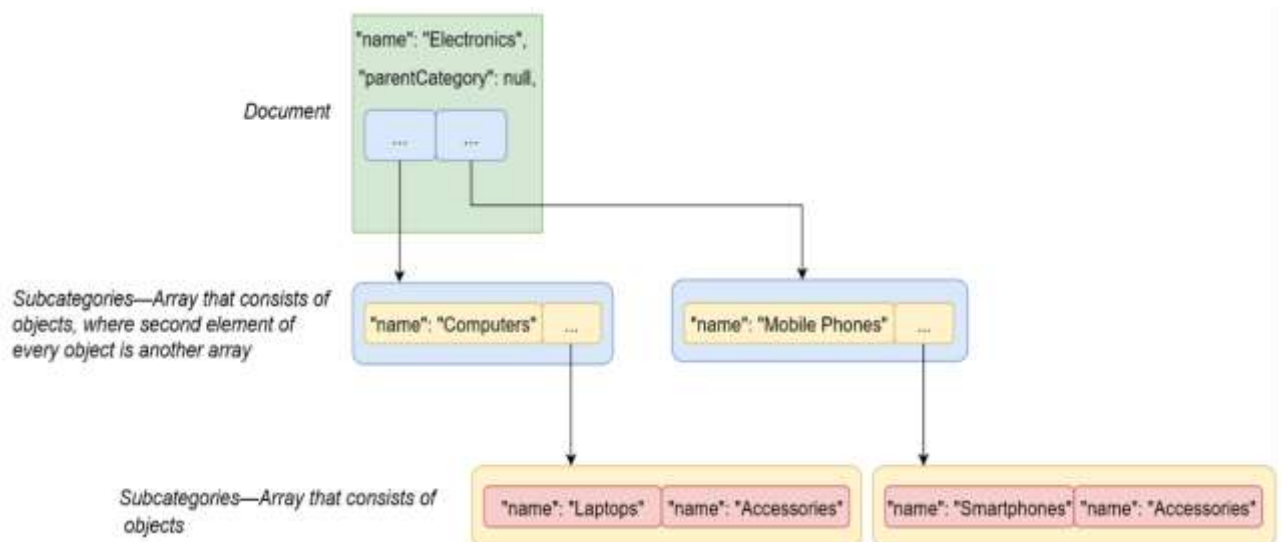
```

```

"parentCategory": null,
"subCategories": [
  {
    "name": "Computers",
    "subCategories": [
      {
        "name": "Laptops"
      },
      {
        "name": "Accessories"
      }
    ]
  }
],
{
  "name": "Mobile Phones",
  "subCategories": [
    {
      "name": "Smartphones"
    },
    {
      "name": "Accessories"
    }
  ]
}
]
}

```

It contains another **subCategories** array, which is a nested array of objects. To understand further,



Nested subcategories inside a MongoDB document

Reading arrays

Basic query operators

The \$in operator

\$in is a query operator used to filter documents based on whether a field's value exists inside the provided array. Simply put, **\$in** means, "Is the value in this list?" It is used inside the **.find()** method when querying documents.

Syntax:

```
db.categories.find({ field: { $in: [value1, value2, ...]}})
```

So, the above query can be rewritten as:

Query:

```
db.users.find({
  "orderHistory.id": { $in: ["3000", "3003", "3004"] }
}, {name:1, orderHistory:1, _id:0})
```

It returns document(s) whose **orderHistory** contains at least one of the listed values.

The \$nin operator

\$nin is a query operator used to filter documents where a field does not match any value in a given array. Simply put, **\$nin** means, "Is the value not in this list?" It is used inside the **.find()** method when querying documents.

Syntax:

```
db.categories.find({ field: { $nin: [value1, value2, ...]}})
```

So, the above query can be rewritten as:

Query: Find users whose order history excludes IDs 3000, 3003, and 3004 using \$nin

```
db.users.find({
  "orderHistory.id": { $nin: ["3000", "3003", "3004"] }
}, {name:1, orderHistory:1, _id:0})
```

It returns document(s) where **orderHistory** contains none of the listed values.

Note: **\$nin** also matches documents where the field doesn't exist at all. For example, the above query will return users with no **orderHistory** at all, e.g., Jason Shields.

Other query operators

Here are the two most commonly used operators: **\$all** and **\$size**.

The \$all operator

Remember, the \$in operator works like an OR condition—it matches documents where a field contains any one of the values in a given array. But what if we want to find documents where a field (array) contains all the specified values?

That's where the \$all operator comes in. It matches documents where the array contains every value we list, regardless of the order.

Syntax:

```
db.categories.find({ field: { $all: [value1, value2, ...] } })
```

For example,

Categories.json:

https://drive.google.com/file/d/1LKvPBMCS_T2o5b0lBmqp-RKD0oOBaFpVR/view?usp=drive_link

Query: Find the categories that include all three subcategories: Educational, Action Figures, and Board Games using \$all

```
db.categories.find({ "subCategories.name": { $all: ["Educational", "Action Figures", "Board Games" ] } })
```

Explanation: The above code finds categories where the subCategories array has these “Educational,” “Action Figures,” and “Board Games” values. Notice that the code returns no output. It means we do not have such a category. Now, remove “Board Games” from the list and rerun. This time, we get a category named “Toy.”

The \$size operator

The \$size operator finds documents where the array contains an exact number of elements.

Syntax:

```
db.categories.find({ field: { $size: n } })
```

For example:

Query: Find the categories that have exactly three subcategories using the \$size operator.

```
db.categories.find({ subCategories: { $size: 3 } })
```

This code finds categories that are further divided into exactly three subcategories. Try experimenting by modifying the value of `$size` to see whether the output changes.

Updating Arrays in Documents

MongoDB allows adding, removing, or modifying values in arrays with the following operations:

- `$push`: Adds a value to an array
- `$set`: Updates a value in an array
- `$unset`: Removes a value from an array

Note: We're using the `.limit()` method at the backend, so we see a truncated neat output automatically. When running locally, make sure to apply the relevant `.find()` method to see the updated results.

The scope of modifications made to the data in this lesson is confined to the executables only. The changes won't reflect elsewhere. We'll see variations of the `.find()` method to read data after those modifications and observe the impact.

The `$push` operator

The `$push` is a special MongoDB operator used to add a new value to an existing array in a document. If the array does not exist, it will automatically create one.

Imagine adding a new subcategory, "Crockery," to the "Home & Kitchen" category from the `categories` collection.

Categories.json:

[https://drive.google.com/file/d/1LKvPBMCS_T2o5b0lBmqp-RKD0oOBaFpVR/view?usp=drive link](https://drive.google.com/file/d/1LKvPBMCS_T2o5b0lBmqp-RKD0oOBaFpVR/view?usp=drive_link)

Query: Add 'Crockery' to the subcategories of 'Home & Kitchen' using `$push`

```
db.categories.updateOne(
  { name: "Home & Kitchen" },
  { $push: { subCategories: { name: "Crockery" } } },
  //Display the change
  db.categories.find({name: "Home & Kitchen"}, {subCategories:1})
```

Explanation: Here, `{ name: "Home & Kitchen" }` is the filter. It finds the category document with the name "Home & Kitchen". Next, the `$push` operator adds a new value to the specified array (`subcategories` in this case). Note that `subcategories.name` is a field within an object in an array, not the array itself. To make this work properly, we need to push an entire object to the array as: `$push: { subcategories: { name: "Crockery" } }`.

The `$set` operator

We have already covered the `$set` operator. Recall the scenario where we wanted to change the address of a user previously. Let's write the correct query.

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query: Update the street of a user from the address array

```
db.users.updateOne(
  { email: "julie.bird362@hotmail.com" },
  { $set: { "address.0.street": "803 Snyder Mills Suite 473" } }
),
// Display the change
db.users.find({ email: "julie.bird362@hotmail.com" }, {name:1, address:1})
```

Explanation:

- `{email: "julie.bird362@hotmail.com"}`: This query finds the first document in the `users` collection where email is "julie.bird362@hotmail.com".
- `{ $set: { "address.0.street": "803 Snyder Mills Suite 473" } }`: This update uses the `$set` operator to update the street. Notice `"address.0.street"`. In every document, `address` is an array consisting of only one object. So, we safely use `address.0`, which refers to the first (and the only) object in the array. Hence, `"address.0.street"` targets the `street` field inside that object.

Note: If an array has multiple objects, and we want to update a field in all of them, we'd need to use the `$set` with `$[.<field_name>]`.

The `$unset` operator

We have already covered the `$unset` operator. Now, let's rewrite the above query, such that we want to remove the street for a specific user.

Query: Remove the street of a user from the address array

```
db.users.updateOne(
  { email: "julie.bird362@hotmail.com" },
  { $unset: { "address.0.street": "" } }
),
// Display the change
db.users.find({ email: "julie.bird362@hotmail.com" }, {name:1, address:1})
```

Explanation:

- `{email: "julie.bird362@hotmail.com"}`: This query finds the first document in the `users` collection where email is "julie.bird362@hotmail.com".
- `{ $unset: { "address.0.street": "" } }`: It specifies to delete the `street` field from the selected document, regardless of its value.

Error Handling in CRUD Operations

Error handling is an essential part of working with any database, including MongoDB. Errors can occur for a variety of reasons:

- Incorrect query syntax
- Violations of database rules (like duplicate IDs)
- Accessing non-existent collections or databases

Understanding how to recognize, handle, and fix these errors can save us time and prevent data loss.

Understanding common errors in MongoDB

MongoDB Shell provides detailed error messages to help us understand what went wrong. They follow a common structure. Below is an example of an error message:

A screenshot of a MongoDB Shell terminal window showing an error message. The text is as follows:

```
1 >
2   "ok": 0,
3   "code": 11000,
4   "errmsg": "E11000 duplicate key error collection: test.users index: _id_ dup key: { _id: 1 }"
5 >
```

Error 11000: Duplicate `_id`—each document must have a unique identifier

Here, `ok: 0` indicates that the operation failed, `code` refers to a specific error code (e.g., 11000 for duplicate key) and `errmsg` contains a detailed description of the error.

Let's look at some of the common errors in detail.

Duplicate key error, i.e., `_id` conflict

Every document in a MongoDB collection has a unique `_id` field. Attempting to insert a document with an existing `_id` will trigger a "duplicate key error". For example:

```
db.users.insertOne({ _id: 1, name: "Alice" })
db.users.insertOne({ _id: 1, name: "Bob" }) // Error: Duplicate Key
```

The first command creates a new user with `_id: 1`. The second command tries to create another user with the same `_id` (1). Because `_id` is a unique identifier, MongoDB prevents this and throws an error.

How to fix it: Ensure that each document has a unique `_id`. Avoid manually setting `_id` unless necessary, as MongoDB can auto-generate it.

Invalid query syntax

This error occurs when our query is written incorrectly. Common causes are:

- Misspelled commands (`db.usres.insertOne` instead of `db.users.insertOne`)
- Incorrect use of operators (`$set` without `$`)
- Missing curly braces `{}` for objects.

For example:

```
// Incorrect: Missing curly braces
db.users.find(name: "Alice")
```

```
// Correct
db.users.find({ name: "Alice" })
```

Let's see how we can fix it.

Accessing non-existent collections or databases

This occurs when we try to perform operations on collections or databases that do not exist. For example:

```
db.nonexistentCollection.find()
```

If the collection does not exist, MongoDB will create it automatically when we try to insert a document. But if we only try to read from it, we will not see any data because the collection does not exist.

Invalid field names or values

This error occurs when we use field names that are not allowed, such as:

- Field names that contain dots (.) unless they're used for nested fields.
- Field names that start with a dollar sign (\$), unless they are used as operators.

For example:

```
// Incorrect: Field name starts with $
db.users.insertOne({ $name: "Alice" })
```

A few things to keep in mind while fixing.

How to fix it: Make sure field names do not start with special characters (\$, .). Use nested objects for organizing the fields.

Querying with invalid data types

This error occurs when we try to query a field using an incorrect data type. For example:

```
//Incorrect: rating is a number, but the query uses a string
db.products.find({ rating: "3.3" })
```

What is the correct way of doing it?

How to fix it: Always query fields using the correct data type:

- **Numbers:** Without quotes (25)
- **Strings:** With quotes ("Alice")
- **Boolean:** true or false

Using non-existent operators

This happens when we use an invalid operator (like set instead of \$set).

For example:

```
//Incorrect: Missing $
db.users.updateOne({ name: "Alice" }, { set: { email: "rhodes@miller.com" } })

// Correct:
db.users.updateOne({ name: "Alice" }, { $set: {email: "rhodes@miller.com" } })
```

Introduction to the Aggregation Framework

Why should we use aggregation?

The aggregation framework in MongoDB is a powerful, built-in data processing toolkit. It lets us perform complex operations such as filtering, grouping, transforming, and calculating directly inside the database. Think of it like creating reports, summaries, or analytics without using another tool or moving the data somewhere else. For example:

- **Data analysis in the database:** We can calculate things like the total number of orders and the average rating.
- **Data transformation:** Suppose our documents contain too much information, and we only want a summary. We can reshape our data, e.g., grouping data or sorting it by a specific value.
- **Better performance:** Instead of fetching a large amount of raw data and processing it in our app, aggregation lets MongoDB do the heavy lifting. This means less data is transferred over the network.

How does aggregation work

The aggregation framework works like an assembly line or conveyor belt called a pipeline. Each document in a MongoDB collection is passed through a sequence of stages, with each stage performing a specific operation. Here are some common stages:

- **Matching:** Filters documents
- **Grouping:** Groups documents and allows us to perform calculations
- **Sorting:** Sorts the results in order
- **Projecting:** Reshapes fields in the output

Each stage takes input, processes it, and sends the result to the next stage.

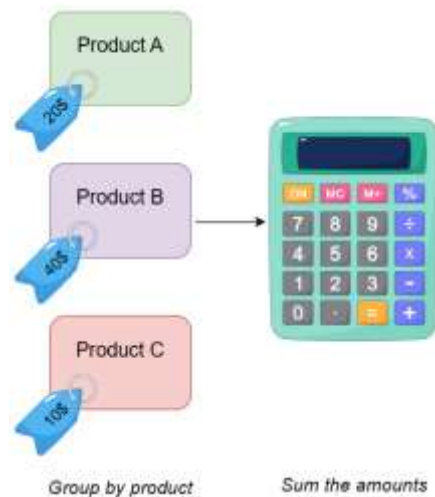
Syntax:

```
db.collection.aggregate([  
  { stage1 },  
  { stage2 }  
])
```

Examples

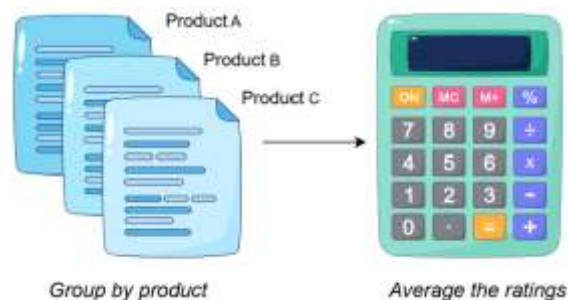
Here are some real-world examples of when aggregation is useful.

Calculate how much money was spent on each product.



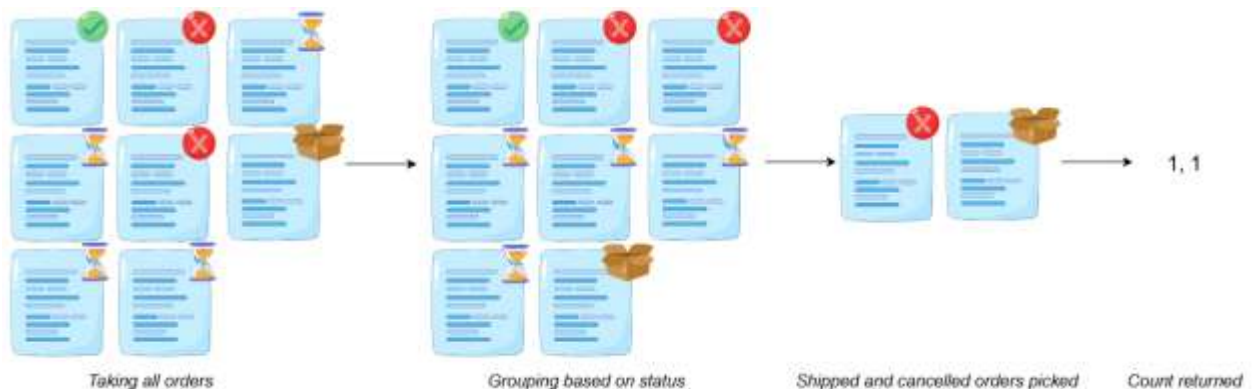
Groups orders by product and sums the spending to show total amount

Calculate the average rating for each product provided in reviews.



Aggregates reviews by product and calculates average ratings

Calculate the total number of shipped and cancelled orders.



Groups orders by status and calculates the total number of shipped and cancelled orders

Aggregation Stages

MongoDB offers some keywords to add stages in our pipelines to perform required actions.

The **\$match** stage

It filters the documents and is very similar to the **.find()** method. Here's an example used to filter documents in the **orders** collection to include only shipped orders.

Query:

```
db.orders.aggregate([
  {
    $match: {
      status: "Shipped"
    }
  }
])
```

Orders.json:

https://drive.google.com/file/d/1iKf1Ne_NHrK0OqAbfYAqV3HV1o_xVa5W/view?usp=sharing

Explanation: In line 1, we directly specify that this query needs to be executed via the aggregation pipeline: **db.orders.aggregate**. Here, **\$match** tells MongoDB to only include documents where the **status** field is exactly **"Shipped"**.

Note: This stage is typically the first step in an aggregation pipeline to reduce the number of documents that need to be processed later.

The **\$sort** stage

Similar to the **.sort()** method, the **\$sort** stage sorts the documents in the pipeline on the basis of a criterion. Here's an example that sorts products by their price:

Products.json:

https://drive.google.com/file/d/1dIFBoZnQaimfy5gh_CEqYijV15LAu8re/view?usp=sharing

Query:

```
db.products.aggregate([
  {
    $sort: { price: 1 }
  }
])
```

Explanation: In line 1, we directly specify that this query needs to be executed via the aggregation pipeline: `db.products.aggregate`. `$sort: {price: 1}` tells MongoDB to sort products on the basis of price in ascending order.

The `$project` stage

Similar to the projection applied in the `.find()` method, the `$project` stage includes/excludes specific fields. Suppose we want to show names of users only.

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query:

```
db.users.aggregate([
  {
    $project: {
      name: 1,
    }
  },
])
```

Explanation: In line 1, we directly specify that this query needs to be executed via the aggregation pipeline: `db.users.aggregate`. `$project: {name: 1}` tells MongoDB to only show the names of users. The `_id` field is also visible because it is not explicitly excluded.

The `$group` stage

It groups documents by a specified field and performs calculations (sum, average, count, etc.). Here's an example to count the number of orders based on their statuses:

Orders.json:

https://drive.google.com/file/d/1iKf1Ne_NHrK0OqAbfYAqV3HV1o_xVa5W/view?usp=drive_link

Query:

```
db.orders.aggregate([
  {
    $group: {
      _id: "$status",    // Group by the status field
      count: { $sum: 1 } // Count how many documents (orders) per status
    }
  }
])
```

Explanation: In **line 1**, we directly specify that this query needs to be executed via the aggregation pipeline: `db.orders.aggregate`. Here, `_id: "$status"` tells MongoDB to group all orders by their `status` value, and adds a new field `count` in the result, which is computed as `{ $sum: 1 }`. This adds 1 for each order in each group, resulting in a total count per status.

Here's another simpler example. Suppose we want to calculate the average rating of products provided in the `reviews` category. The following query would do:

Reviews.json:

https://drive.google.com/file/d/1LCKiYUQPq9ThPWhWm4P4s2pXktDJ7L9v/view?usp=drive_link

Query:

```
db.reviews.aggregate([
  {
    $group: {
      _id: "$productId",
      averageRating: { $avg: "$rating" },
      totalReviews: { $sum: 1 }
    }
  }
])
```

Explanation: Let's break it down. In **line 1**, we directly specify that this query needs to be executed via the aggregation pipeline: `db.reviews.aggregate`. Here, `$group` groups all review

documents by `productId`, calculates the average of the `rating` field using `$avg`, and counts the number of reviews using `$sum: 1`.

With these aggregation stages, we're now prepared to build insightful pipelines—enabling advanced analytics and a deeper understanding of our MongoDB data.

Multi-stages Aggregation in Action

What is a multi-stage pipeline?

A **multi-stage pipeline** is a sequence of aggregation stages, each transforming the data and passing it to the next stage. These stages are executed in order, and the output of one stage becomes the input for the next, allowing us to build complex data transformations step by step, much like an assembly line.

Let's begin creating multi-stage pipelines for the real-world examples we covered earlier.

Examples

For instance, if we need to calculate the number of orders that got cancelled and shipped, we can use the aggregation pipeline. Take a look at the query below.

Orders.json:

https://drive.google.com/file/d/1iKf1Ne_NHrK0OqAbfYAqV3HV1o_xVa5W/view?usp=drive_link

Query:

```
db.orders.aggregate([
  { $match: {
    status: { $in: ["Shipped", "Cancelled"] } }
  },
  { $group:
    { _id: "$status",
      count: { $sum: 1 }
    }
  }
])
```

Let's break it down. The task can be divided into two stages:

1. Filter the document(s) from the `orders` collection where `status` is `"Shipped"` or `"Cancelled"`.
2. Count the total number of shipped orders and cancelled orders, separately.

Note that the second stage is dependent on the first stage, i.e., the output of the first stage serves as the input to the second stage.

In **line 1**, we directly specify that this query needs to be executed via the aggregation pipeline: `db.orders.aggregate`. As mentioned above, this aggregation has two stages: `$match` and `$group`.

- `$match`: This stage filters the `orders` collection to only include documents where the `status` field is either `"Shipped"` or `"Cancelled"`. Any other statuses are ignored.
- `$group`: This groups the filtered documents by their status value. For each group, it:
 - Uses the value of status (like `"Shipped"` or `"Cancelled"`) as the `_id`.
 - Counts how many documents are in that group by adding 1 for each document with `$sum: 1`.

In short, this pipeline answers the question: “How many orders are shipped and how many are cancelled?” And, it does so efficiently, right inside the database, without needing to fetch and process the data elsewhere.

Here's another example. Let us sort users by the number of orders in `orderHistory`. Remember that sorting by the number of array items always requires two stages in a pipeline.

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query: /// Error in the query

```
db.users.aggregate([
  // Step 1: Add a new field that counts number of orders
  {
    $addFields: {
      orderCount: { $size: "$orderHistory" }
    }
  },
  // Step 2: Sort users by order count in descending order
  {
    $sort: {
      orderCount: -1
    }
  }
])
```

What happened? Why does this generate an error, when `orderHistory` is an array? This is because, for some users, there exists no order history. To avoid this error, wrap `$orderHistory` in `$ifNull`, so missing fields are treated as empty arrays.

Query: /// One way to overcome the error

```
db.users.aggregate([
  // Step 1: Add a new field that counts number of orders
  {
    $addFields: {
      orderCount: { $size: { $ifNull: ["$orderHistory", []] } }
    }
  },
  // Step 2: Sort users by order count in descending order
  {
    $sort: {
      orderCount: -1
    }
  }
])
```

Explanation: In line 1, we directly specify that this query needs to be executed via the aggregation pipeline: `db.users.aggregate`. There are two stages in this pipeline:

- First, `$addFields` adds a new field `orderCount`, which is the length of the `orderHistory` array, using `$size`. `$ifNull: ["$orderHistory", []]` means if `orderHistory` is missing or `null`, treat it as `[]`.
- Then, `$sort` arranges the documents in descending order (-1) by `orderCount`.

We can't write: `db.users.aggregate([ { $sort: { { $size: "$orderHistory" } : -1 } } ])`, because the expression `{ $size: "$orderHistory" }` is not valid inside `$sort`, as it expects direct field names, not computed expressions.

Here's another trick. Instead of `$addFields` stage, we can also use `$project` stage for the above query:

Query: /// Another way to overcome the error

```
db.users.aggregate([
  {
    $project: {
      orderCount: { $size: { $ifNull: ["$orderHistory", []] } }
    }
  },
])
```

```
{
  $sort: {
    orderCount: -1
  }
}
```

To calculate how much money was spent on each product across all orders, we can use an aggregation pipeline as follows:

1. Unwind the `items` array to access each product individually.
2. Group by `productId` to aggregate all occurrences of the same product and calculate the total spent per product using `$sum` on `quantity * price`.

Let's see the query below:

Orders.json:

https://drive.google.com/file/d/1iKf1Ne_NHrK0OqAbfYAqV3HV1o_xVa5W/view?usp=drive_link

Query:

```
db.orders.aggregate([
  // Step 1: Unwind the items array
  {
    $unwind: "$items"
  },

  // Step 2: Group by productId and calculate total spent
  {
    $group: {
      _id: "$items.productId",
      totalSpent: {
        $sum: {
          $multiply: ["$items.quantity", "$items.price"]
        }
      }
    }
  }
])
```

Explanation: Let's break it down. In **line 1**, we directly specify that this query needs to be executed via the aggregation pipeline: `db.orders.aggregate`. As mentioned above, this aggregation has two stages: `$unwind` and `$group`.

- **\$unwind**: It turns each document with multiple items into multiple documents—one per item. This allows us to work with each product entry separately.
- **\$group**: `_id: "$items.productId"` groups the documents by each unique product. For each group, **\$sum** adds up the total spent per product:
 - Multiplies `quantity * price` for each item
 - Then adds them up for the same product

Note: Notice that we can add as many stages as needed, such as **\$skip**, **\$limit**, or additional **\$project** stages for further transformation. To know more about stages, refer to the [complete list of aggregation operators](#).

Grouping Data Across Collections

Up until now, the scope of our queries has been confined to one collection at a time. But as the data is spread across multiple collections, we may need to write queries that fetch data from more than one collection simultaneously. Suppose we need to find the product(s) bought by a specific user. To do this, we first filter the required user and retrieve their order history from the **users** collection. Then, using that order history, we fetch the products purchased in each order from the **orders** collection.

MongoDB doesn't allow grouping across collections in a single **\$group** stage directly, but we can combine (or join) collections first, then group them. Here are the supported methods:

- Use **\$lookup** to join collections.
- Use **\$unionWith** to merge similar collections.

Use **\$lookup** to join collections

We can use **\$lookup** to combine data from another collection before grouping. It's safe to use **\$lookup** when collections are related. For example, in the use case above, we can observe how **users** and **orders** are related to each other via one field, **orderHistory**. The **orderHistory** is an array of order IDs, and in the **orders** collection, we have documents, one for each order. Each document contains details like those shown below.

Query:

```
{
  "_id": {
```

```

      "id": "3001"
    },
    "userId": {
      "id": "0010"
    },
    "orderDate": "2024-11-21T06:32:47.383235",
    "items": [
      {
        "productId": {
          "id": "2014"
        },
        "quantity": 3,
        "price": 67.69
      },
      {
        "productId": {
          "id": "2011"
        },
        "quantity": 3,
        "price": 246.99
      }
    ],
    "totalAmount": 626.83,
    "paymentMethod": "Credit Card",
    "status": "Cancelled"
  }
}

```

Let's devise an aggregation pipeline for this use case.

Orders.json:

https://drive.google.com/file/d/1iKf1Ne_NHrK0OqAbfYAqV3HV1o_xVa5W/view?usp=drive_link

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query:

```

db.users.aggregate([

  // Step 1: Match the specific user by name
  {
    $match: { name: "Allison Hill" }
  },

```

```

// Step 2: Break down the orderHistory array into individual documents
{
  $unwind: "$orderHistory"
},

// Step 3: Join with orders collection using matching order ID
{
  $lookup: {
    from: "orders",           // target collection
    localField: "orderHistory.id", // user's orderHistory ID
    foreignField: "id",       // actual order ID field in orders
    as: "orderInfo"
  }
},

// Step 4: Flatten the joined orderInfo array
{
  $unwind: "$orderInfo"
},

// Step 5: Flatten the items array inside each order
{
  $unwind: "$orderInfo.items"
},

// Step 6: Select only the user name and productId from each order item
{
  $project: {
    _id: 0,
    productId: "$orderInfo.items.productId"
  }
}
})

```

Explanation:

- The **\$match** stage selects the document for "Allison Hill" from the **users** collection. This step narrows the pipeline to only one user, so all subsequent operations apply only to her.
- The **\$unwind** stage then breaks the **orderHistory** so that each order is processed individually.
- Next, the **\$lookup** stage tells MongoDB to pull documents from the **orders** collection.
 - **localField: "orderHistory.id"** specifies a field from the current collection (**users**) to match against. Because we previously used **\$unwind** on **orderHistory**, we're now accessing the embedded field **id** of a single order entry.

- **foreignField: "id"** points to the field in the **orders** collection where MongoDB should look for a match. So it tries to match **"orderHistory.id"** from **users** with **"id"** in **orders**.
- **as: "orderInfo"** is the name of the new field to be created in the pipeline's output. It is an array containing the matching order(s) from **orders**.
- **orderInfo** is an array of matching orders returned from the **\$lookup**. Even if there's only one match, it's still in array form. The next **\$unwind** stage flattens that array, so each document now contains one order object instead of an array.
- Within each order (now a single object), there's an **items** array—representing the products included in that order. The next **\$unwind** stage flattens the **items** array, so each document represents one item from that order.
- The final **\$project** stage displays the product names bought by the specific user, filtered in the first step of aggregation.

Let's see another example: Which products received poor reviews (**rating < 3**), and what are their names? In this case, the purpose of aggregation pipeline is to:

1. Search the **reviews** collection for all reviews where **rating < 3**.
2. For each such review, use **\$lookup** to fetch the corresponding product from the **products** collection using **productId**.
3. Return the names and ratings of the selected products.

The following query solves this problem:

Reviews.json:

https://drive.google.com/file/d/1LCKiYUQPq9ThPWhWm4P4s2pXktDJ7L9v/view?usp=drive_link

Products.json:

https://drive.google.com/file/d/1dIFBoZnQaimfy5gh_CEqYijV15LAu8re/view?usp=sharing

Query:

```
db.reviews.aggregate([
  // Step 1: Filter reviews where rating is less than 3
  {
    $match: {
      rating: { $lt: 3 }
    }
  }
])
```

```

    }
  },

  // Step 2: Join with products collection using productId
  {
    $lookup: {
      from: "products",
      localField: "productId", // in reviews: productId is a string
      foreignField: "id",      // in products: id is a number
      as: "productInfo"
    }
  },

  // Step 3: Unwind the productInfo array to flatten it
  {
    $unwind: "$productInfo"
  },

  // Step 4: Project only desired fields
  {
    $project: {
      _id: 0,
      productId: 1,
      productName: "$productInfo.name",
      rating: 1
    }
  }
])

```

Explanation:

- The `$match` stage filters the `reviews` collection to include only reviews where the `rating` is less than 3. These are considered poor or negative reviews. This is the starting point of the pipeline.
- Because `$lookup` returns an array (`productInfo`), this `$unwind` stage flattens it into a single object. This allows us to directly access fields inside the product document (like `productInfo.name`).
- This `$project` stage reshapes the output. It hides the default `_id` field and shows the `productName`.

Merge Similar Collections with \$unionWith

The `$unionWith` operator in MongoDB is a powerful feature that allows us to combine the results of multiple collections into a single aggregation pipeline. This is especially useful when

we have multiple collections with similar schemas and want to analyze or report on them as if they were a single dataset.

Setting up

Up until now, our database does not contain similar collections. Let's build up a use case for this. The tech team has decided to keep cancelled orders in a separate collection, instead of keeping them in the `orders` collection. The following query does this:

Query:

```
db.orders.aggregate([          // Picking cancelled orders from orders collection
{
  $match: {
    status: "Cancelled"
  }
},
{
  $merge: {
    into: "cancelledOrders", // Adding cancelled orders to a new collection
    whenMatched: "keepExisting",
    whenNotMatched: "insert"
  }
}
]),

db.orders.deleteMany({        // Deleting cancelled orders from orders collection
  status: "Cancelled"
}),

db.cancelledOrders.find().limit(5) // Printing first 5 orders from the new collection
```

Explanation:

- `$match` filters only orders with status `"Cancelled"`.
- `$merge` writes those documents into a new collection called `cancelledOrders`. If it doesn't exist, MongoDB will create it automatically, which is true in our case.
 - `whenMatched: "keepExisting"` controls what happens when a document with the same `_id` already exists in the target collection. `keepExisting` means, "If a document with the same `_id` already exists in `cancelledOrders`, do not overwrite it—keep the original." This is a safe default when moving data and wanting to avoid accidental overwrites.

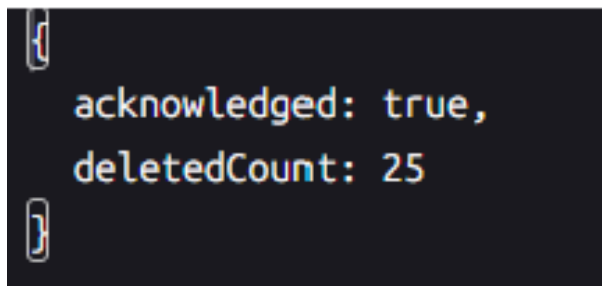
- `whenNotMatched: "insert"` controls what happens when there is no document with the same `_id` in the target collection. `insert` means, "Go ahead and insert the new document into `cancelledOrders`."

Now that we've safely copied these documents to `cancelledOrders`, we can delete them from the original collection (see **lines 16–18**):

Query:

```
db.orders.deleteMany({
  status: "Cancelled"
})
```

This removes all documents where `status === "Cancelled"` from the `orders` collection. We can see the following acknowledgement in the output.



```
{
  acknowledged: true,
  deletedCount: 25
}
```

Merging collections

Imagine a scenario where we're asked to get all orders (both cancelled and active) and calculate the total revenue. The objective is to combine all documents from both `orders` and `cancelledOrders` collections and compute the total revenue by summing up the `totalAmount` field.

NewOrders.json:

[https://drive.google.com/file/d/13HEyeQk2kg4BHbXYYQiUG2DBWpaW_CgV/view?usp=drive link](https://drive.google.com/file/d/13HEyeQk2kg4BHbXYYQiUG2DBWpaW_CgV/view?usp=drive_link)

cancelledOrders.json:

https://drive.google.com/file/d/16L3bsZdQrHaVUbtOeYGHWC_eavEkb7ON/view?usp=sharing

Query:

```
db.orders.aggregate([
  { $unionWith: "cancelledOrders" },
  {
    $group: {
      _id: null,
      totalRevenue: { $sum: "$totalAmount" }
    }
  }
])
```

Here, { `$unionWith: "cancelledOrders"` } merges the documents from two collections:

- The pipeline starts with `orders` as the base collection.
- `$unionWith` brings in all documents from `cancelledOrders`.

After this stage, the pipeline contains a combined dataset of all documents from both collections. Next, the `$group` stage groups documents together (because `_id: null` means “group everything into one bucket”), and calculates the total value of the `totalAmount` field across all orders.

With `$unionWith`, we can seamlessly merge similar collections, enabling unified analysis and reporting across distributed datasets.

Introduction to Indexing

What is an index?

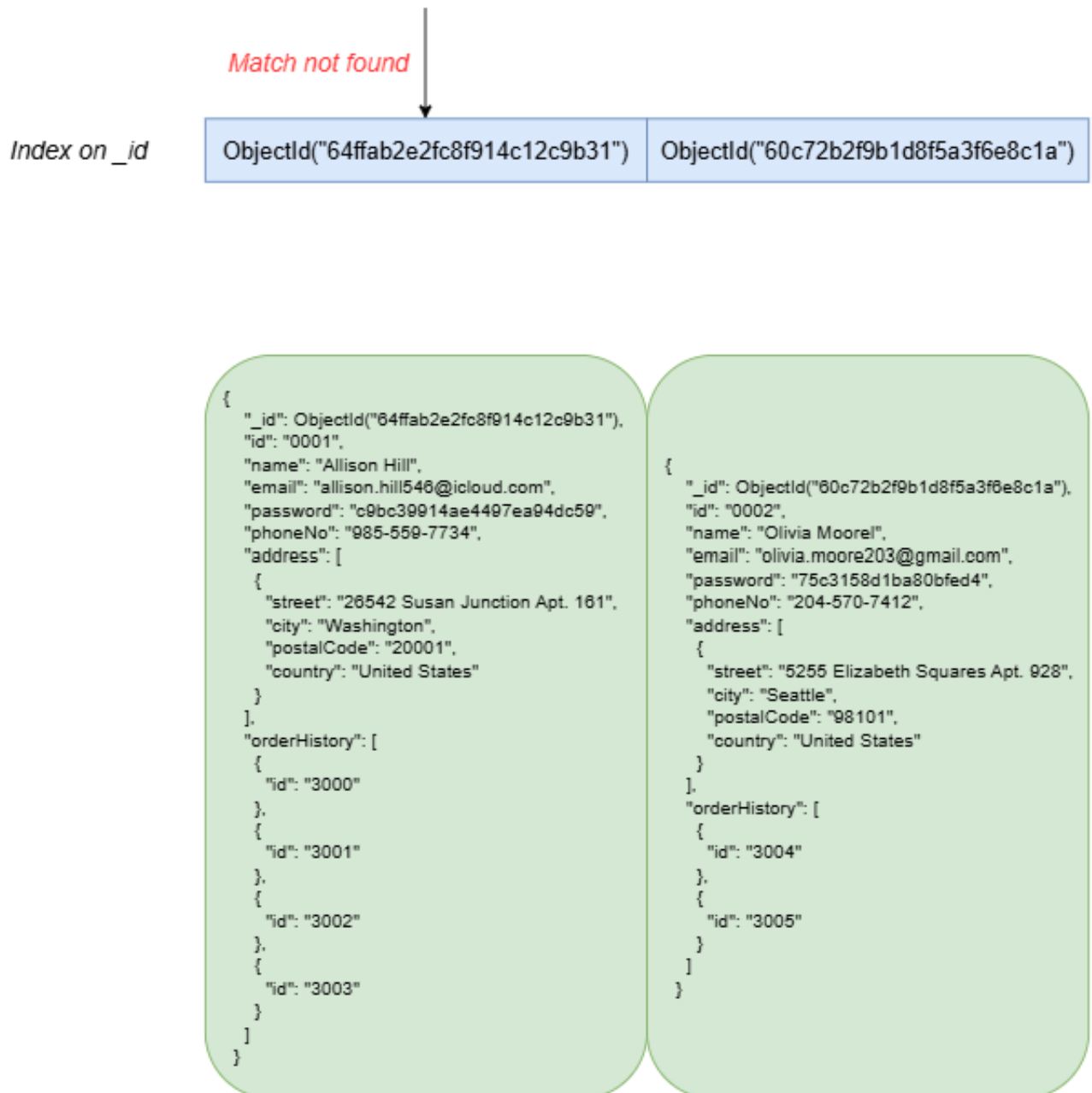
An **index** is a data structure that improves the speed and efficiency of data retrieval operations. It acts like a roadmap for the database engine, allowing it to quickly locate documents without scanning the entire collection.

Imagine trying to find a specific topic in a large book. Without an index, we'd need to flip through every page. With an index, we can jump directly to the right page. In MongoDB, indexes function similarly; they store a subset of the data in an optimized format to make lookups faster.

Default index

Every MongoDB collection automatically includes an index on the `_id` field. This guarantees fast access when querying by document ID. This index is created by default, ensures each document's uniqueness, and allows for fast, efficient lookups.

If indexes didn't exist, querying against `_id` would require MongoDB to search every document line by line to find a match. But with indexing, it can directly jump to the required document. See the illustration below, where the search is optimized when queried against the following `_id` value: `ObjectId("60c72b2f9b1d8f5a3f6e8c1a")`.



Match found

Index on `_id`

ObjectId("64ffab2e2fc8f914c12c9b31")

ObjectId("60c72b2f9b1d8f5a3f6e8c1a")

...

```
{
  "_id": ObjectId("64ffab2e2fc8f914c12c9b31"),
  "id": "0001",
  "name": "Allison Hill",
  "email": "allison.hill546@icloud.com",
  "password": "c9bc39914ae4497ea94dc59",
  "phoneNo": "985-559-7734",
  "address": [
    {
      "street": "26542 Susan Junction Apt. 161",
      "city": "Washington",
      "postalCode": "20001",
      "country": "United States"
    }
  ],
  "orderHistory": [
    {
      "id": "3000"
    },
    {
      "id": "3001"
    },
    {
      "id": "3002"
    },
    {
      "id": "3003"
    }
  ]
}
```

```
{
  "_id": ObjectId("60c72b2f9b1d8f5a3f6e8c1a"),
  "id": "0002",
  "name": "Olivia Moorel",
  "email": "olivia.moore203@gmail.com",
  "password": "75c3158d1ba80bfed4",
  "phoneNo": "204-570-7412",
  "address": [
    {
      "street": "5255 Elizabeth Squares Apt. 928",
      "city": "Seattle",
      "postalCode": "98101",
      "country": "United States"
    }
  ],
  "orderHistory": [
    {
      "id": "3004"
    },
    {
      "id": "3005"
    }
  ]
}
```

Index on `_id`

ObjectId("64ffab2e2fc8f914c12c9b31")	ObjectId("60c72b2f9b1d8f5a3f6e8c1a")	..
--------------------------------------	--------------------------------------	----

Quick indexing



If that's the case, why do we need additional indexes? If we're querying by any other field, e.g., email or name, the `_id` index is no longer helpful. In that case, MongoDB will do a complete collection scan.

Additional indexes

Assume users are logging into the e-commerce web app. When a user logs in, they provide their email address and password. To authenticate, our backend executes this query:

```
db.users.find({email:"<email id>" })
```

This happens every time a user logs in. MongoDB will perform a full collection scan to find the user document, even if there are millions of users. This slows the response time and isn't scalable as the user base grows.

To resolve this issue, we can add additional indexes based on how our app actually queries the data. For example, we can create an index on the email field.

<i>Index on _id</i>	ObjectId("64ffab2e2fc8f914c12c9b31")	ObjectId("60c72b2f9b1d8f5a3f6e8c1a")
<i>Index on email</i>	"allison.hill546@icloud.com"	"olivia.moore203@gmail.com"

<pre>{ "_id": ObjectId("64ffab2e2fc8f914c12c9b31"), "id": "0001", "name": "Allison Hill", "email": "allison.hill546@icloud.com", "password": "c9bc39914ae4497ea94dc59", "phoneNo": "985-559-7734", "address": [{ "street": "26542 Susan Junction Apt. 161", "city": "Washington", "postalCode": "20001", "country": "United States" }], "orderHistory": [{ "id": "3000" }, { "id": "3001" }, { "id": "3002" }, { "id": "3003" }] }</pre>	<pre>{ "_id": ObjectId("60c72b2f9b1d8f5a3f6e8c1a"), "id": "0002", "name": "Olivia Moorel", "email": "olivia.moore203@gmail.com", "password": "75c3158d1ba80bfed4", "phoneNo": "204-570-7412", "address": [{ "street": "5255 Elizabeth Squares Apt. 928", "city": "Seattle", "postalCode": "98101", "country": "United States" }], "orderHistory": [{ "id": "3004" }, { "id": "3005" }] }</pre>
--	--

Indexing on email generated

Here, MongoDB builds a structure behind the scenes. Now, it can quickly locate a specific email, without scanning irrelevant entries.

Why are indexes important?

Below are some advantages of indexing.

- **Boosts query speed:** Especially for large datasets, indexed queries execute much faster.
- **Reduces resource load:** Indexes minimize CPU and disk I/O by avoiding full collection scans.

- **Supports sorting and uniqueness:** Indexes can maintain order for sort operations, and unique indexes enforce data integrity by preventing duplicate entries.

Creating and Removing Single-Field and Compound Indexes

Creating single field indexes

A single field index is created on one field and is most useful when that field is frequently queried.

Syntax:

```
db.users.createIndex({ fieldName: 1 }) // 1 = ascending, -1 = descending
```

1 or -1 represents the order of sorting. 1 actually indexes documents in ascending order of the `fieldName`; -1 indexes them in descending order of the `fieldName`.

When we use `.sort()` in a MongoDB query, MongoDB can either:

1. Sort the data in memory after fetching (as seen in earlier lessons). It is slower, especially for large datasets.
2. Use an index that is already sorted on that field.

Using an index to support sorting avoids unnecessary in-memory sorting. For example, in the previous lesson, we talked about a scenario where indexing on the `email` field can help us speed up the authentication process.

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query: Searching email and view execution stats

```
db.users.find({ email: "debra.gibbs523@icloud.com"}).explain("executionStats")
```

Here, the `explain("executionStats")` method helps us see what's actually happening. It inspects how MongoDB runs a query. Here are the key highlights:

- 300 documents were scanned.
- 1 document was returned.

- No index was used.
- Execution time is 0ms, but could increase as the dataset grows.

The following index accelerates queries that search by `email`.

Query: Create an ascending index on the email field

```
db.users.createIndex({ email: 1 }),  
db.users.find({ email: "debra.gibbs523@icloud.com"}).explain("executionStats")
```

Here are the key highlights:

- 1 document was examined.
- 1 document was returned.
- The `email_1` index was used.
- Execution time is 1ms.

Note that MongoDB automatically names indexes based on their key pattern. The name is derived from the field and sort order. For example, with `{email: 1}` key pattern we get the `email_1` index. Now, try changing the key pattern to `{email: -1}` in the code above. This time we will get the `email_-1` index.

Missing field or null values

Creating a (regular) index on `email` does *not* require every document to have an `email` field. So, when we run `db.users.createIndex({ email: 1 })`, MongoDB will index only the documents that have an `email` field and ignore documents where `email` is missing or null. So it's perfectly fine if some users have no email—they just won't appear in the index.

Enforcing uniqueness

Creating a regular index doesn't ensure uniqueness. It means that two or more users may have the same email address. What if we want to create a unique index so no two users can have the same email? Take a look at the query below.

Query: Create a unique index on the email field to prevent duplicate entries

```
db.users.createIndex({ email: 1 }, { unique: true })
```

The above indexing ensures that no email is repeated. Now, MongoDB requires that all values in the index be unique.

If two documents have the same value against a field, it will raise a duplicate key error. This means that if documents have a missing field or explicit null values, there will still be a conflict because MongoDB treats missing values as duplicates. For example, let's create an index on `orderHistory` from the `users` documents.

Query: Create a unique index on the orderHistory field to prevent duplicate entries

```
db.users.createIndex({ orderHistory: 1 }, { unique: true })
```

To allow some documents not to have `orderHistory` and still enforce uniqueness where it exists, we can do:

Query: Create a unique, sparse index on orderHistory to enforce uniqueness only on documents that have the field

```
db.users.createIndex({ orderHistory: 1 }, { unique: true, sparse: true })
```

This query creates an index, making sure that documents with `orderHistory` must be unique. Documents with `null` values and those without `orderHistory` are excluded from the index. Here, with `{orderHistory: 1}` key pattern we get the `orderHistory_1` index. Now, try changing the key pattern to `{orderHistory: -1}` in the code above. This time we will get the `orderHistory_-1` index.

Creating compound indexes

Compound indexes span multiple fields. They are ideal for queries involving multiple conditions.

For example, customers are likely to use the following filters in combination:

- Category (e.g., "Electronics", "Clothing")
- Price (e.g., products under \$500)

Products.json:

https://drive.google.com/file/d/1dIFBoZnQaimfy5gh_CEqYijV15LAu8re/view?usp=sharing

Query: Find "Electronics" products under \$500 and sort them by price in ascending order

```
db.products.find({ category: "Electronics", price: { $lt: 500 } }).sort({ price: 1 }).explain("executionStats")
```

The above query filters the data but requires a full collection scan. Here are the key highlights:

- 100 documents were examined.
- 6 documents were returned.
- No index was used.
- Execution time is 0ms, but may increase as the volume of data grows.

In order to speed up the queries, especially when the product catalog grows to thousands of items, we can add a compound index against this use case:

Query: Create a compound index on category and price, and analyze performance with .explain('executionStats')

```
db.products.createIndex({category: 1, price: 1}),  
db.products.find({ category: "Electronics", price: { $lt: 500 } }).explain("executionStats")
```

Here are the key highlights:

- 6 documents were examined.
- 6 documents were returned.
- No full collection scan was done.

Single vs. compound indexes

Use case	Index type
Filter/sort on one field.	Single field
Filter/sort on multiple fields.	Compound index
Optimize for read-heavy queries.	Compound index

Removing indexes

To remove (drop) an index in MongoDB, we can use a method called `dropIndex()`. It deletes the specified index from a collection.

Syntax:

```
db.collection.dropIndex({ field1: 1, field2: -1 })
```

Here, `{ field1: 1, field2: -1 }` represents the key pattern used to create the index. For example, look at the query below.

Users.json:

<https://drive.google.com/file/d/1ncRwdsIrzPniamKOHXnHBxaN-5tmCxbY/view?usp=sharing>

Query:

```
// Single and compound indexes created
db.products.createIndex({category: 1, price: 1}),
db.users.createIndex({ email: 1 }),
// Dropping indexes
db.products.dropIndex({category: 1, price: 1}),
db.users.dropIndex({ email: 1 })
```

Explanation: In lines 2–3, a single and a compound index are created, respectively. In lines 6–7, `dropIndex()` is used to drop the indexes by adding the exact same pattern.

Introduction to Data Modelling

Discover the basics of data modeling in MongoDB, focusing on how to structure data using embedded and referenced documents for better performance and scalability.

What is data modeling?

Data modeling is the process of organizing and structuring data so it aligns with the application's needs. It involves:

- Defining the types of data to store (like users, products, orders).
- Determining how the data is stored (fields, data types, embedded or separate documents).
- Specifying how different pieces of data are related.

In MongoDB, data modeling means designing documents (which are JSON-like objects) and organizing them into collections, similar to how relational databases use rows and tables. A good data model is like a solid foundation for a building—it affects everything built on top of it. For example:

- **Performance:** If we model our data to match how it's queried, MongoDB can fetch it with fewer operations. For instance, embedding frequently accessed data reduces the need for aggregation or multiple queries.
- **Maintainability:** A clear, logical structure makes it easier for developers to understand, update, and debug the database. Well-named fields and predictable structures improve collaboration.
- **Scalability:** As our app grows (more users, more data), a scalable model ensures the database remains fast and manageable. Proper indexing and document sizing help avoid performance bottlenecks.

Note: MongoDB uses a flexible schema, meaning documents in the same collection can look different. However, it's still best practice to keep them as consistent as possible to avoid confusion and ensure easier querying.

Example: Modeling a user

In our database, we have multiple collections: `users`, `products`, `reviews`, `categories`, and `orders`. Let's talk about how we modeled `users`.

We should embed related information into a single document, which is precisely what we did for `users`. Embedding means storing related data inside a single document. For example:

```
{
  "id": "0001",
  "name": "Allison Hill",
  "email": "jillrhodes@miller.com",
  "passwordHash": "hashed_password_1",
  "address": [
    {
      "street": "908 Jennifer Squares",
      "city": "Robinsonshire",
      "postalCode": "36964",
      "country": "Sudan"
    }
  ],
  ...
}
```

Sample user document in MongoDB with personal details and an array of address objects

Why did we embed? Because user-related data (like addresses) is closely tied to the user and usually fetched together—for example, when displaying a profile or processing an order. Embedding makes reads faster by avoiding joins or additional queries, which is ideal for such tightly coupled data.

Embedded vs. referenced documents

Here's a question: “If the user places lots of orders, and each order has its own details, should we embed order information in the user document?” The answer is, “No, use referenced documents.” Referencing means storing related data in separate documents, linked by an identifier. It's ideal to use referenced documents when related data is large, updated independently, or occasionally read.

For example, there's an array `orderHistory` in almost every user document, as follows:

```
"orderHistory": [
  {
    "id": "3000"
  },
  {
    "id": "3001"
  },
  {
    "id": "3002"
  },
  {
    "id": "3003"
  }
]
```

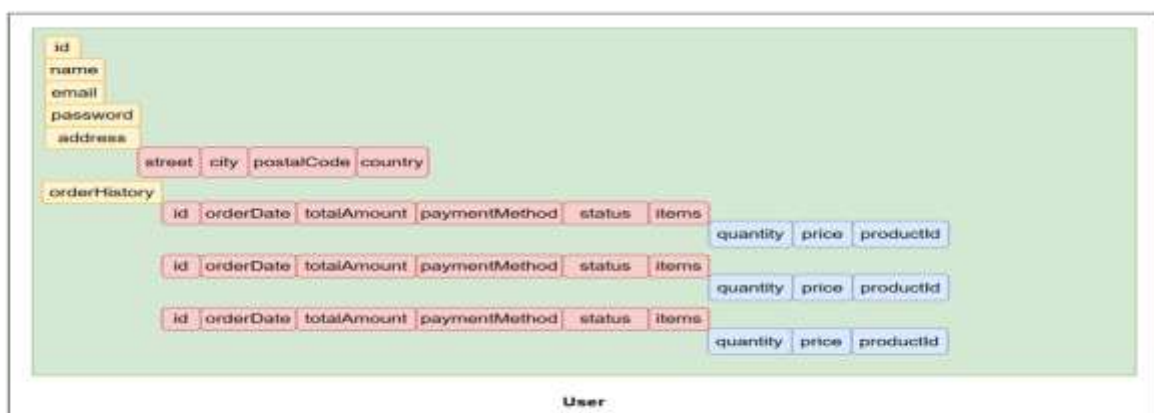
Sample orderHistory array showing multiple order IDs associated with a user

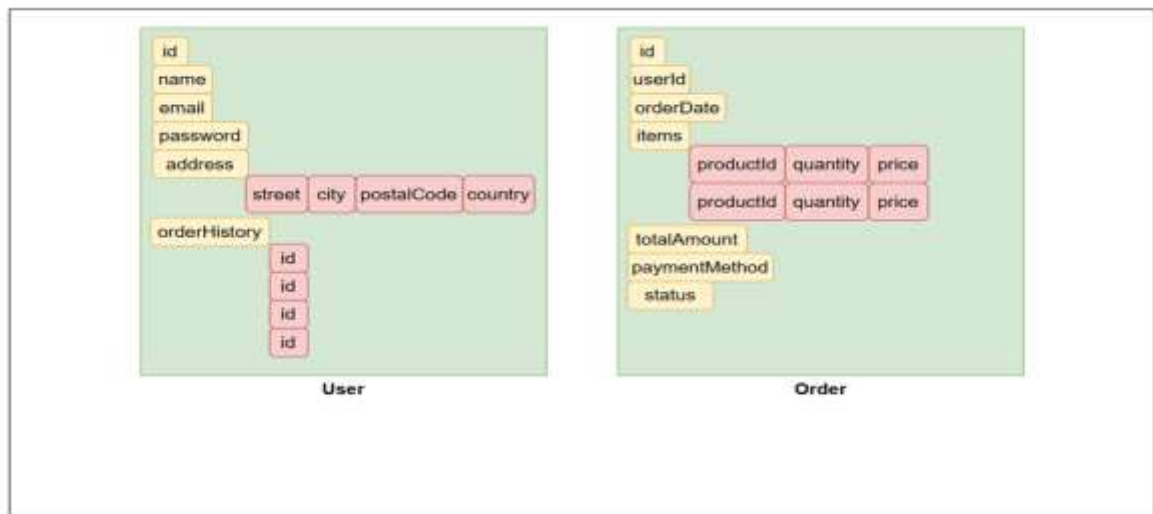
Whereas, information of every order is stored in the **orders** collection separately. Look at the sample document below.

```
{
  "id": "3001",
  "userId": "0001",
  "orderDate": "2024-11-21T06:32:47.383235",
  "items": [
    {
      "productId": 2049,
      "quantity": 3,
      "price": 103
    },
    {
      "productId": 2083,
      "quantity": 3,
      "price": 46
    }
  ],
  "totalAmount": 447,
  "paymentMethod": "Credit Card",
  "status": "Cancelled"
}
```

Sample order document with user ID, order details, item list, and payment status

Rather than embedding complete order details inside the user document, we use referencing. This avoids oversized documents, keeps data manageable, and allows orders to be updated or queried independently. See the illustration below.





The table below summarizes the comparison between embedded and referenced documents.

Strategy	Description	When to use
Embedded	Nest related data inside a parent document.	We frequently access both together (e.g., product + reviews).
Referenced	Store IDs pointing to related documents in another collection.	Data is reused in many places or is too large to embed.

Thoughtful data modeling in MongoDB—whether through embedding or referencing—lays the basis for efficient, maintainable, and scalable applications that grow seamlessly with our data and user base.

Data Validation in MongoDB—JSON Schema

Think of data validation as MongoDB's way of checking our homework—it makes sure we're turning in work that follows the rules. Without it, our database might end up with:

- Typos in field names.
- Wrong data types (like a string instead of a number).

- Missing required fields.

This can lead to issues if our app depends on reliable data. With validation, MongoDB acts like a gatekeeper, only allowing data that matches our defined rules.

What is JSON schema validation?

A JSON schema is a way to describe what our data should look like:

- What fields are required?
- What type of data should each field contain?
- Can a field only accept certain values?

MongoDB uses this schema to *validate every document* inserted into or updated in a collection.

Example: Creating a collection with validation

Suppose we want every user to have a `name` (string) and an `email` (string):

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email"],
      properties: {
        name: { bsonType: "string" },
        email: { bsonType: "string" }
      }
    }
  }
})
```

Create a users collection with schema validation requiring name and email as strings

What is this code doing? Let's break it down.

- `db.createCollection("users", {...})`: This tells MongoDB to create a new collection named `users`.
- `validator: {...}`: This part defines a validation rule for documents in the collection. It ensures that every document added to `users` follows a specific structure and data types.

- **\$jsonSchema: {...}**: MongoDB uses the JSON Schema standard to define validation rules. Everything inside this block is part of the schema that checks the structure and content of documents.
- **bsonType: "object"**: This says that each document must be a BSON object (like a JSON object). In other words, we can't insert just a string or a number—it has to be a structured document like: `{ "name": "Alice", "email": "alice@example.com" }`.
- **required: ["name", "email"]**: This means that every document should have both **name** and **email** fields. If either is missing, MongoDB will reject the document.
- **properties: {...}**: This is where we define specific rules for each field in the document.
- **name: { bsonType: "string" }**: The **name** field must exist and be a string.
- **email: { bsonType: "string" }**: The **email** field must exist and be a string.

If we try to insert or update a document (by removing a required field or not adding a required field) that doesn't follow the schema, MongoDB will reject the operation and return an error.

Now, if we run an insert command like:

```
db.users.insertOne({ "name": "Bob", "email": "bob@example.com" })
```

MongoDB will check if the document still matches the schema. In this case, it does—both **name** and **email** are still present and correctly typed. So, the insertion is valid and will succeed.

But, if we try an insert command like:

```
db.users.insertOne({ "name": "Bob" }) // missing "email"
```

MongoDB will block this update. Because email is a required field and is missing, it causes the document to violate the schema. This is considered an invalid update to the collection.

Try the above creation and insertion commands in the shell below.

Let's see another scenario. Imagine that the administration needs to revise the schema that stores product information in MongoDB. To avoid bad or incomplete data, your task is to enforce a schema that every product must follow. Each product document must:

- Be an object.
- Contain the following required fields:
 - **name** (must be a string)
 - **price** (must be a number)
 - **inStock** (must be a boolean)
- Optionally include:
 - **tags** (must be an array of strings, if present)

The following query would work.

```
db.createCollection("products", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "price", "inStock"],
      properties: {
        name: { bsonType: "string" },
        price: { bsonType: "number" },
        inStock: { bsonType: "bool" },
        tags: {
          bsonType: "array",
          items: { bsonType: "string" }
        }
      }
    }
  }
})
```

Create a products collection with schema validation using \$jsonSchema to enforce field types and requirements.

Look at **lines 10–12**. This validation rule defines the expected structure of the **tags** field. **bsonType: "array"** indicates that **tags** should be an array, and **items: { bsonType: "string" }** means that every element inside the array must be a string.

Benefits of schema validation

The following are a few benefits of schema validation:

- Prevents accidental mistakes and typos
- Helps keep our data consistent
- Makes debugging and maintenance easier

By using the validation, MongoDB helps us avoid human errors. We do not need to check for missing or badly formatted data later, as our database stays predictable and reliable.

Best Practices for Schema Design

Discover essential MongoDB data modeling tips, including designing for queries, embedding related data, maintaining document size, ensuring consistency, and using indexes for better performance.

A thoughtful model helps your application run faster, stay flexible, and scale easily. Instead of just thinking, “How should I store this data?” ask, “How will I use this data in my app?” This mindset leads to practical, performant, and maintainable data models. The following are some tips to follow.

Tips for schema design

Let's have a look at the tips:

Design for queries

“Don't just model our data around how it *looks*—model it based on how it will be *used*.” Think in terms of real-world use cases. Design your documents based on how your app fetches and updates data.

Querying is often the most frequent and performance-critical part of an application. If your schema mirrors your access patterns, MongoDB can return results faster and with fewer operations. Let's say our app frequently needs to show user profiles along with their last five orders. Rather than making multiple queries, we can serve this view with a single document fetch with the following better schema:

```
{  
  "name": "Alice",
```

```
"email": "alice@example.com",
"recentOrders": [
  { "id": "ord1", "total": 42.50, "date": "2024-04-05" },
  { "id": "ord2", "total": 18.00, "date": "2024-04-03" },...
],
...
}
```

Sample user document with basic information and a recentOrders array containing order details

Favor embedding for the related data

Choose between embedding and referenced documents wisely.

“If two types of data are always accessed together, consider embedding them.”

Embed related data when it's small and accessed together. We can embed related data that doesn't grow unbounded and updates across both datasets are needed. Use references when data is large or shared across documents.

Keep documents reasonably sized

“MongoDB documents have a size limit (currently 16 MB)—but best practice is to keep them *much smaller*.” Avoid unbounded arrays, deeply nested structures that could grow too large, and embedding rarely-used data.

Use pagination strategies, e.g., store only recent data or split into pages. Break data into multiple documents if it grows too large. Think of a MongoDB document like a single file. It's better to have several organized folders (collections/documents) than one massive, messy file.

Use consistent field names and types

“Inconsistent schemas make querying harder and break our indexes.” Stick to a naming convention and use the same types for the same fields across documents.

Consider having different conventions for the same field, e.g.:

```
{ "userId": "123" }
{ "user_id": "124" }
{ "userid": 125 }
```

Inconsistent field naming can lead to data retrieval issue

These inconsistencies force us to handle multiple cases in code and weaken indexing.

Add indexing for efficient queries

“Indexes are like signposts for our database—they help MongoDB quickly find data without scanning every document in a collection.”

Index fields, we frequently search or sort on. As we know, by default, MongoDB performs a collection scan—checking every document to find matches. This becomes slow as your data grows. Indexes allow MongoDB to narrow the search just like how the index at the back of a book helps us jump straight to the right page.

Without an index, queries slow down as the number of documents increases and app may suffer performance issues under load. Whereas, with an index, queries return results faster and MongoDB uses fewer resources to fetch data.

Use schema validation

“Even though MongoDB doesn't require a fixed schema, we *can and should* enforce structure where appropriate.”

As shown in the [previous lesson](#), JSON Schema validation helps enforce structure and prevent bad data. It prevents insertion of documents with missing or invalid fields and catch typos and type errors early. For example:

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "email"],
      properties: {
        name: { bsonType: "string" },
        email: { bsonType: "string" },
      }
    }
  }
});
```

Create a users collection with JSON schema validation to ensure name and email are required strings

Now, inserting a user without **name** or **email** will fail.

Bonus tips for real-world projects

Here are some bonus tips that can help us when working on a real-world project:

- Avoid using **null** and **undefined** inconsistently—pick one for missing data.

- Test your schema with realistic sample data before going live.
- Document your schema for your team (especially if it's flexible!)

We need to remember one golden rule, i.e., “Model our data based on how our app queries it, not how it looks in theory.”

In short, designing MongoDB schemas requires balancing flexibility and discipline. While we're free to store documents any way we like, these best practices help ensure your database remains easy to query, resistant to bugs, and available as your app grows.

User Roles and Permissions in MongoDB

Learn to control access in MongoDB using role-based access control, where users are assigned roles with specific permissions to protect data integrity, security, and collaboration.

Why do we control access?

Imagine every employee can change the prices or delete customer data—that's a recipe for disaster.

In databases, controlling access protects our:

- **Data integrity:** It prevents unauthorized changes.
- **Security:** It keeps sensitive information safe.
- **Collaboration:** It ensures team members only have access to what they need.

MongoDB provides a flexible, role-based access control system to help with this.

What are users and roles in MongoDB?

A user is an account that connects to a MongoDB database. Each user can be assigned one or more roles. A role is a set of permissions that defines what the user is allowed to do. MongoDB comes with several built-in roles that cover common use cases.

Role	What it allows
read	Can only read data from a database
readWrite	Can read and write data

dbAdmin	Can run administrative commands like creating indexes
userAdmin	Can manage users and their roles
root	Has full access to all operations and databases

We can assign different roles to different users based on what they need to do. For example:

- **Customer support agent:** Needs to read customer orders
- **Content manager:** Needs to update products
- **DBA (Database Admin):** Needs to manage indexes
- **Security admin:** Manages user accounts

Creating a user with a role

Let's say we want to create a user who can only read data from the `ecommerce` database (no writing or deleting). Here's how we'd do it:

```
db.createUser({
  user: "readonly",
  pwd: "password123",
  roles: [ { role: "read", db: "ecommerce" } ]
})
```

Create a read-only user for the `ecommerce` database using `db.createUser`

In the snippet above, `readonly` is the username. `password123` is the password for logging in. The user is assigned the `read` role, which limits them to read-only access on the `ecommerce` database. Try the above command in the following terminal:

Note: Once users and roles are defined, we can enable authentication and authorization in MongoDB. We'll see this in the upcoming lesson.

Before enabling authentication and authorization, let's add some more users/roles to our database. Suppose the administration needs to secure the database by adding multiple users, each assigned different roles as described below:

- Create a user, `superadmin`, with full access.
- Create a user, `developer`, who can read and write data.
- Create a user, `auditor`, who can only read data.
- Create a user, `manager`, who can manage users.

```
db.createUser({
  user: "superadmin",
  pwd: "securepass",
  roles: ["userAdminAnyDatabase", "readWriteAnyDatabase"]
})
```

```
db.createUser({
  user: "developer",
  pwd: "devpass",
  roles: [{ role: "readWrite", db: "project" }]
})
```

```
db.createUser({
  user: "auditor",
  pwd: "auditpass",
  roles: [{ role: "read", db: "project" }]
})
```

```
db.createUser({
  user: "manager",
  pwd: "managerpass",
  roles: [{ role: "userAdmin", db: "project" }]
})
```

Create multiple users with different roles and privileges across databases using `db.createUser`

By assigning the right roles to the right users, MongoDB helps enforce least-privilege access—strengthening security, promoting accountability, and enabling safer collaboration across teams.

Authentication, Authorization, and Common Errors

Understand the difference between authentication (verifying user identity) and authorization (controlling user actions) in MongoDB, and how to enable authentication, create users, and handle common errors.

Authentication verifies the identity of a user (e.g., username and password). Whereas, authorization determines what actions an authenticated user is allowed to perform (based on roles and permissions). Authentication keeps unauthorized users out, and authorization limits what authenticated users can do.



Summary of the process

How MongoDB handles authentication

By default, MongoDB may run without authentication (not recommended for production). We can enable authentication so users must log in before accessing the database. To enable authentication, we need to perform the following steps. Switch to the **admin** database to create an administrative user.

```
mongosh admin
```

Note: Global roles like `userAdminAnyDatabase`, `readWriteAnyDatabase`, and `dbAdminAnyDatabase` are defined only in the admin database.

```
db.createUser({
  user: "superadmin",
  pwd: "securepass",
  roles: [ { role: "userAdminAnyDatabase", db: "admin" }, "readWriteAnyDatabase" ]
})
```

Create a superadmin user with full admin rights and read/write access to all databases
superadmin has permission to manage users and access all databases.

Now, exit the shell and then stop the MongoDB server:

Exit

Restart it with the `--auth` flag:

```
mongod --dbpath /data/db --shutdown
```

```
mongod --auth --dbpath /data/db --bind_ip_all --logpath /var/log/mongod.log --fork
```

Command "`mongod --dbpath /data/db --shutdown`" stops the MongoDB server by pointing to the same `--dbpath` used when it was started. It ensures the server shuts down cleanly before restarting with new options (like authentication). Whereas, "`mongod --auth --dbpath /data/db --bind_ip_all --logpath /var/log/mongod.log --fork`", restarts the MongoDB server with authentication enabled.

- `--auth` enforces login for all users.
- `--bind_ip_all` allows connections from any IP (use with caution in production).
- `--logpath` specifies where logs are saved.
- `--fork` runs the server as a background process.

Authentication is now enabled. Because authentication is on, we now must log in as a user:

```
mongosh -u "superadmin" -p "securepass" --authenticationDatabase "admin"
```

We are now logged in as the admin user. By default, the connection opens the `test` database. To begin working with the `ecommerce` database, switch to it using the `use ecommerce` command.

Now, let's create all the other users that we did earlier in the [previous lesson](#):

```
db.createUser({
  user: "developer",
  pwd: "devpass",
  roles: [{ role: "readWrite", db: "ecommerce" }]
})
```

```
db.createUser({
  user: "auditor",
  pwd: "auditpass",
  roles: [{ role: "read", db: "ecommerce" }]
})
```

```
db.createUser({
  user: "manager",
  pwd: "managerpass",
```

```
roles: [{ role: "userAdmin", db: "ecommerce" }]
})
```

Create developer, auditor, and manager users with specific roles in the ecommerce database

How MongoDB handles authorization

Now, let's test what each user can or can not do. Try logging in as each user in a new terminal and test the following actions.

As a developer

```
mongosh -u "developer" -p "devpass" --authenticationDatabase "ecommerce"
```

Try inserting a document

```
use ecommerce
```

```
db.products.insert({ name: "Build feature X" }) //Should work
```

Try creating a new user:

```
db.createUser({ user: "test", pwd: "123", roles: [] }) //Should fail
```

As a manager

```
mongosh -u "manager" -p "managerpass" --authenticationDatabase "ecommerce"
```

Try creating a user:

```
use ecommerce
```

```
db.createUser({
  user: "intern",
  pwd: "internpass",
  roles: [{ role: "read", db: "ecommerce" }]
}) // Should work
```

Create a read-only intern user in the ecommerce database after switching to it.

Try inserting a new document:

```
db.tasks.insert({ name: "Assign tasks" }) // Should fail
```

Note: To stay updated on any changes, we can use

the `db.getUsers()` and `db.getRoles()` commands to inspect users and roles.

How do they look? Try the above commands in the following terminal:

Error handling and debugging

MongoDB provides detailed error messages for failed operations. Understanding these messages helps us quickly identify and fix problems. Common error scenarios are listed below:

- **Validation errors:** This occurs when a document doesn't meet schema requirements. For example, trying to insert a user without an email field (see [here](#)).
- **Authentication errors:** This occurs when a user provides incorrect credentials. For example, Authentication failed.
- **Authorization errors:** This occurs when a user tries to perform an action they don't have permission for, as seen above. For example, not authorized on `ecommerce` to execute the command.

Read the error message carefully—it usually tells us what went wrong. Check the schema validation rules if we see validation errors. Verify user roles and permissions for authorization issues.